UNIVERSITETI METROPOLITAN TIRANA

FACULTY OF COMPUTER SCIENCE AND IT

COMPUTER ENGINEERING DEPARTMENT

MASTER OF SCIENCE IN ARTIFICIAL INTELLIGENCE WITH PROFILE: CYBERSECURITY

DIPLOMA THESIS

TITLE: DESIGN AND SIMULATION OF AN AI-DRIVEN EXTENDED DETECTION AND RESPONSE (XDR) SYSTEM

DEAN OF THE FACULTY	Prof. Asoc. Dr. Ligor NIKOLLA
HEAD OF DEPARTMENT	Dr. Elton DOMNORI
SUPERVISOR	Dr. Igli HAKRAMA
CANDIDATE	Darling SELITA

TIRANA 2026

Declaration of Authenticity

I, Darling Selita, hereby declare that I wrote this thesis on my own and did not use any unnamed sources or aid. Thus, to the best of my knowledge and belief, this thesis contains no material previously published or written by another person except where due reference is made by correct citation. This includes any thoughts taken over directly or indirectly from printed books and articles as well as all kinds of online material. The work contained in this thesis has not been previously submitted for examination.

Date __ / __ / _____ Signature _____

List of Abbreviations

AI - Artificial Intelligence

ML - Machine Learning

XDR - Extended Detection and Response

EDR - Endpoint Detection and Response

SIEM - Security Information and Event Management

SOAR - Security Orchestration, Automation and Response

IDS - Intrusion Detection System

IPS - Intrusion Prevention System

TTP - Tactics, Techniques, and Procedures

NIST - National Institute of Standards and Technology

NIST CSF - NIST Cybersecurity Framework

ISO - International Organization for Standardization

ISMS - Information Security Management System

CIA - Confidentiality, Integrity, and Availability

LLM - Large Language Model

APT - Advanced Persistent Threat

IP - Internet Protocol

CDN - Content Delivery Network

CICIDS - Canadian Institute for Cybersecurity Intrusion Detection System

XAI - Explainable Artificial Intelligence

RQ - Research Question

UML - Unified Modeling Language

Abstrakt

Teknologjia ndodhet në një nga periudhat më kulminante të saj, duke sjellë zhvillim dhe përmirësim të vazhdueshëm në të gjitha fushat e jetës sonë të përditshme. Por, me rritjen e varësisë ndaj sistemeve digjitale apo të automatizimeve, duhet të merret parasysh dhe siguria e tyre, në mënyrë që këto sisteme të ofrojnë shërbim të mirë dhe të vazhdueshëm, duke e bërë kështu, sigurinë e të dhënave një sfidë të madhe për mbarëvajtjen.

Sistemet tradicionale të përdorura, si SIEM, EDR, fokusohen kryesisht tek parandalimi i këtyre sulmeve, duke skanuar herë pas herë log-et e sistemit, në mënyrë që të kuptojë se kur një aktivitet është normal apo i dyshimtë, duke kërkuar gjithashtu një ndërhyrje njerëzore për të kuptuar se çfarë ka ndodhur dhe si duhet përgjigjur. Këtu hyn në punë XDR, një sistem i cili synon të integrojë fjalën e fundit të teknologjisë, Inteligjencën Artificiale, dhe nëpërmjet një baze të qendërzuar të të dhënave nga ku të gjithë kompanitë, të cilat shesin shërbime për mbrojtjen kibernetike, jo vetëm marrin të dhëna por edhe e përditësojnë këtë bazë të dhënash në rast të një virusi apo anomalie të re të shfaqur në sistemet e tyre përkatëse, të automatizojë të gjithë procesin e kundërpërgjigjes ndaj sulmeve apo incidenteve.

Kjo temë propozon zhvillimin e një sistemi simulues XDR, të bazuar në Inteligjencë Artificiale, i cili do ketë si qëllim jo vetëm analizimin e log-eve të sistemit, por edhe marrjen e një vendimi autonom për kundërpërgjigje. Ky sistem XDR do të përdorë Machine Learning, për të trajnuar si të identifikohen sjellje jo-normale, si dhe të simulohen veprime për të izoluar dhe bllokuar një proces apo një instancë të dyshimtë.

Qëllimi përfundimtar, përtej simulimit, do të jetë vlerësimi i efikasitetit dhe funksionalitetit të kësaj mënyre të të punuarit, për të vlerësuar më pas gjithashtu dhe besueshmërinë, saktësinë dhe sigurinë e vendimmarrjes së këtij sistemi apo çdo lloj sistemi inteligjent.

Fjalë kyçe: *Inteligjencë Artificiale, Siguri e të dhënave, SIEM, EDR, XDR, Machine Learning, Analizimi i log-eve, Sisteme digjitale, Vendimmarrje autonome.*

Abstract

Technology is in one of its most culminating periods, bringing continuous development and improvement in all areas of our daily lives. But, with the increasing dependence on digital systems or automation, their security must also be taken into account, so that these systems provide good and continuous service, thus making data security a major challenge for the well-being.

Traditional systems used, such as SIEM and EDR, focus mainly on preventing attacks by scanning system logs periodically, in order to understand when an activity is normal or suspicious, also requiring human intervention to understand what happened and how to respond. This is where XDR comes into play—a system that aims to integrate the latest technology, Artificial Intelligence, and through a centralized database from which all companies that sell cyber protection services not only receive data but also update this database in the event of a new virus or anomaly appearing in their respective systems, to automate the entire process of responding to attacks or incidents.

This thesis proposes the development of an XDR simulation system, based on Artificial Intelligence, which will aim not only to analyse system logs, but also to make an autonomous decision to respond. This XDR system will use Machine Learning to train how to identify abnormal behaviour, as well as simulate actions to isolate and block a suspicious process or instance.

The ultimate goal, beyond simulation, will be to evaluate the efficiency and functionality of this way of working, to then also assess the reliability, accuracy, and security of decision-making of this system or any type of intelligent system.

Keywords: *Artificial Intelligence, Data Security, SIEM, EDR, XDR, Machine Learning, Log Analysis, Digital Systems, Autonomous Decision-Making.*

Table of Contents

Table of Figures

List of Tables

Chapter 1 – INTRODUCTION

In modern times, technology hasn't been stopping one bit, and has kept climbing the steps of evolution rapidly, therefore transforming our everyday life, and keeping us in touch with it for our day-to-day activities. Not only automated systems, but also digital systems and big complex infrastructures, that tech companies use to operate their work environment on, have been deeply embedded with data collection & processing, communication and service delivery. However, these advancements come with their own problems, one of which—if not the most important one—is the cybersecurity threats that these infrastructures face every day.

As more organizations, not only in IT or Cybersecurity especially, depend on these customs, there is also a big need of insurance for these services, not only that they function well, but also that their service is uninterrupted, and the data being used is correct and not always tampered with, as well as available to the instances who need it. Therefore, Cybersecurity has become a crucial component to keeping these systems and infrastructures running at all times and without errors.

1.1 Problem Statement

Cybersecurity solutions that are mostly used as of now are SIEMs (Security Information Event Management) and EDRs (Endpoint Detection Services), to monitor system logs and detect potential suspicious activity, or potential threats. This is done continuously, thus giving these systems a hand when comparing normal and abnormal activity.

However, despite their good traits and effectiveness, these solutions present limitations for the current rate of threats that systems face. They sometimes require cyber threat analysts to be involved in the decision-making process, in order to better analyse alerts and investigate these incidents, thus leading to a considerable delay in response time and an increase in the amount of work from these human experts. But as mentioned above, these limitations are currently present, and if we're thinking ahead, there needs to be consideration for the advancements of these threats, of the ways they infiltrate the system, and how hard they hit.

1.2 Motivation

With the problem being apparent, and having a considerable impact on security, XDR is not only a solution to the problem, but also a stepping stone for the next era of incident response systems, coupled with the advancements of AI, offering a better range of support and detection abilities, as well as more automated responses that greatly reduce alert lateness. It is also important to mention that the data that these systems will be fed with will not be only new data, or only old data, but a variation that will help the AI model to have a better sense of direction

when determining whether a log is suspicious or not. The costs of developing these solutions also greatly overextend the cost of human labour, making it a better choice for the future.

1.3 Research Questions

The main goal of this thesis is to find out how effective AI and powerful ML algorithms are when integrated in real-world Extended Detection and Response Systems (XDRs). The concrete questions raised in this thesis are:

RQ1: Can an Artificial Intelligence-driven XDR System accurately identify suspicious logs or events using a centralized telemetry system along with continuous log monitoring and analysis?

RQ2: How much can Machine Learning algorithms improve threat detection, event correlation and risk mitigation?

RQ3: Is it possible to simulate an almost fully automated incident response mechanism with an AI + ML Engine, while maintaining the level of reliability and accuracy that normal XDR solutions offer?

RQ4: How effective is intrusion detection and event correlation in early mitigation of sequential or multi-scale attacks?

1.4 Research Hypotheses

The previous section considered all the questions posed and where the research will be based, regarding the main points of where the research will be conducted. However, there need to be hypotheses raised, in order to properly evaluate them during the implementation and testing part of this thesis's product.

H1: An Artificial Intelligence-driven XDR system can identify suspicious logs and security events through telemetry collection, continuous monitoring and automated AI-driven analysis with a good accuracy level.

H2: The integration of Machine Learning algorithms greatly improves the effectiveness of an XDR solution compared to current systems that mostly use rule-based mechanisms.

H3: Event correlation and Intrusion Detection (ID) can identify threats that launch large scaled attacks throughout the whole network, affecting all devices.

1.5 Aim and Objectives

The main objective of this thesis is to design and develop a miniature version of an XDR system, based on Artificial Intelligence and Machine Learning. To destructurize this objective, the following course of action is:

- Analyse existing Cybersecurity solutions (SIEM, EDR, SOAR).
- Go through Machine Learning techniques to find what is best suited to detecting suspicious activity.
- Design a system capable of pulling and analysing data from a data source, as well as develop an AI model to detect anomalies and return the appropriate response such as target isolation, blocking and process killing.
- Evaluate the metrics of this system, to determine if it is reliable and safe to be applied on a much larger scale.

1.6 Methodological Review

The first part of the study involves an overview of existing Cybersecurity Solutions, AI-based detection techniques and XDR systems, found in academic reviews and research as well as industry standards such as NIST or ISO.

The next part will cover the design and development of the proposed XDR system, with the focus of improving the existing ones. Based on the review, there will need to be an assessment of the correct technologies and tools needed to develop this system, to not only be functional, but if successful, even scalable, with focus on data processing quality and model selection.

This step will also involve finding the appropriate datasets to train the model with, once selected, to then apply ML algorithms to detect suspicious behaviour and simulate needed responses.

After all of that, the last step will be to thoroughly analyse the system, using performance metrics such as detection accuracy, response effectiveness towards solving the problem, reliability and availability. This will also help when drawing conclusions, in order to determine any setbacks that will need further research or development.

Chapter 2 – LITERATURE REVIEW

This chapter represents a view of the existing literature for Cybersecurity solutions and defence systems, in regard to the evolution of complexity and intensity of cyber threats. As the systems that people use every day continue to scale based on their needs, the requirement for good security to prevent unauthorized access to these systems becomes more crucial.

The primary parts will include an analysis of SIEM, EDR and SOAR systems, how they work and how effective they are with the current range of cyber threats, followed by the turning points that an XDR system will offer to solve existing problems or fatigue in regard to anomaly detection and resolution.

This will be done by also expanding on the role of Artificial Intelligence and Machine Learning in enhancing such systems and building upon them, to improve these mechanics.

Finally, the chapter will identify limitations and problems of current approaches, that also show the research gap, which in turn requires the innovation and development of systems such as XDRs.

2.1 Cybersecurity Basics and Threat Environment

Big companies and organizations rely heavily on digital systems or platforms, which they use to offer their users their services. In the background, all these systems and platforms are run through data collection and interpretation subsystems, applied for communication, data storage and service delivery (CDNs). However, this also brings upon the issue of security, integrity and availability of these subsystems, as they are crucial to maintaining these platforms operational and running.

Cyber threats have evolved almost at the same pace as technology advancements, thus making it difficult to maintain a level of security within the infrastructure. According to the MITRE ATT&CK framework [1], phishing (T1566) remains one of the most predominant attacks, occurring daily and directed to every endpoint of the infrastructure. The low cost – high reward matrix makes it a very favourable decision for the attacker to use and establish a persistent connection with the victim, unknowingly.

However, behind these statistics lies a vast amount of information regarding techniques, tactics and procedures, that helps cyber threat analysts better understand these attacks and how to interpret the alerts that a SIEM or an EDR show.

The NIST framework [2] gives a clear overview of what an event is, how events are categorized, and how to know what to be aware of. It also emphasizes the need for incident response mechanisms, including protecting data (personal or business), as well as keeping integrity of

the systems. The proposed XDR has the scope of doing so in a shorter amount of time, also keeping precision and correctness during the decision-making process.

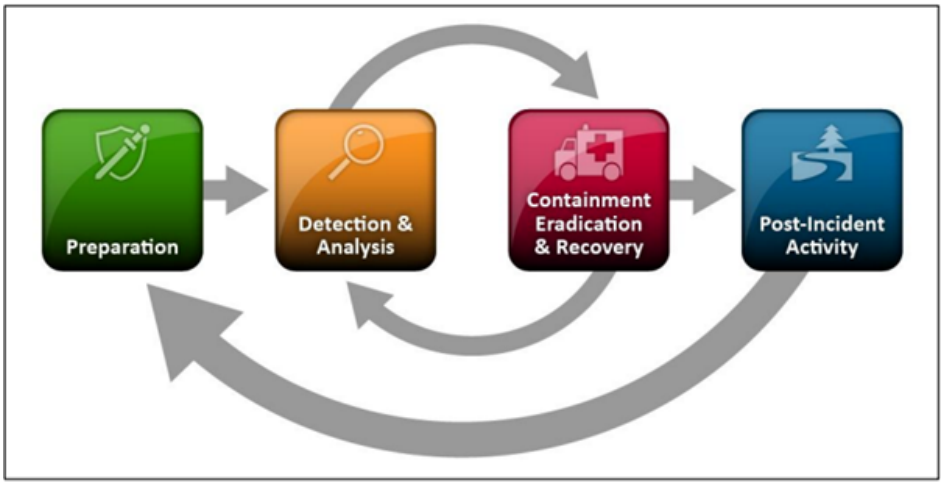

Figure 2.1: Incident Response Lifecycle (NIST.SP.800-61r2)

The other problem that needs to be addressed is the increase of better set-out attacks, that require a more complex course of action needed to be taken in order to defend and respond. Even with the use of the MITRE ATT&CK framework, which documents new methods, the responses are still delayed and slow due to interpretation or understanding of issues.

2.2 Traditional Detection Systems: SIEM and EDR — Procedure & Limitations

Corporate security monitoring and compliance projects have traditionally started with Security Information and Event Management (SIEM) solutions. The main purpose of these systems is to collect log data and perform continuous checks, as well as work in parallel with other cybersecurity solutions like IDS (Intrusion Detection Systems) and IPS (Intrusion Prevention Systems) from which log data is also taken. Most SIEMs use a centralized language, translating the logs from their native language into one unified language [3].

Figure 2.2: Evolution of Threat Detection [3]

SIEMs have been used for years, but as noted before, the technology advancements have made it even more difficult to keep up with new and sophisticated threats. SIEMs remain a good choice for overall management, coupled with a very insightful dashboard in most cases, but when a fast-paced attack occurs, these systems may not be able to appropriately deliver a response in the required amount of time, resulting in loss of data, resources and time.

On the other hand, Endpoint Detection & Response (EDR) systems are concentrated on identifying and reacting to the unique threats discovered on endpoints (end-users or workers of a company/organization), giving a much more focused insight into endpoint activities, monitoring, detection and appropriate response. Due to better visibility from an EDR, cyber threat analysts are more prone to using it on a day-to-day basis [4].

However, EDR offers a more centralized view of the data but is limited to a single endpoint, or a small number of endpoints, making it harder to manage a large number of endpoints in real time. This is the case for most organizations, which have many devices—not just computers, but servers and CDNs as well—thus bringing out the limitations of this technology.

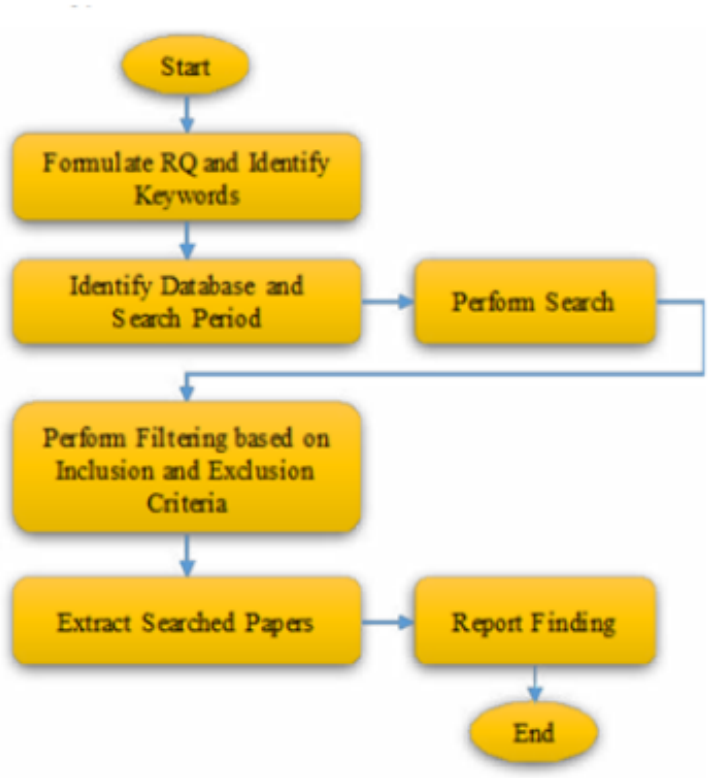

Figure 2.3: EDR Work Methodology [4]

2.3 Extended Detection and Response (XDR)

To reiterate the previously mentioned problem: the cyber threats of today are much more complex and scaled, and with traditional systems you can only achieve so much protection against them. On these grounds, with the recent developments of AI and cross-layer visibility, a new security approach has been established—the Extended Detection and Response system, XDR—showing a significant improvement in threat visibility, detection accuracy and emergence when responding to an incident.

An XDR is not the kind of system that should be deployed everywhere, as an EDR and SIEM can work together just fine if the architecture is not complex and the system is not easily accessible. An XDR handles information from various sources, much like a SIEM, collects all these logs, and then initiates a specific response based on what the logs convey. Sources of information include endpoints, networks, cloud architectures and so on. The effectiveness of this system is higher than an EDR or a SIEM by themselves, because of the correlation between different sources of information and layers, making it have a better understanding of security events and enabling faster detection of threats.

According to Microsoft, an XDR solution is designed to unify the work of two or more separate systems into one, reducing the complexity of having to manage and maintain different security tools [5].

One of the advantages of these systems is the ability to automate their actions and responses based on analytics and previous data, which they take through large and public data sets, and then use to continuously update and train their machine learning algorithms [6]. This way, things like isolating compromised endpoints, blocking malicious processes or events, restricting access to the network, or even fully blocking a specific endpoint are things that now do not require any manual intervention or predefined rules.

Another important source of information for XDR systems is the MITRE ATT&CK framework, a globally accessible knowledge base of adversary tactics and techniques, based on real-world observations [7]. It provides structured information about TTPs (Techniques, Tactics and Procedures) that the XDR can use to classify and correlate malicious behaviour throughout system nodes and layers of the architecture.

Despite this, it should be taken into account that an XDR solution does not automatically fix any shortcomings that an organization might have. It is still an imperfect concept, with most XDR systems still relying on predefined sets of rules, having issues with the correlation between different logs from different sources due to training limitations, and most importantly, still potentially requiring human intervention in the case of unique alerts or final response decisions.

As cybersecurity threats become more sophisticated, the second generation of XDR systems began to incorporate AI and machine learning technologies altogether. These advancements allowed for better anomaly detection, predictive analytics and automated response capabilities. XDR systems could now learn from data, and directly apply the knowledge learned in response to an incident [6].

2.4 Artificial Intelligence and Machine Learning in Cybersecurity

The increasing difficulty of having to face complex and frequent cyberattacks has raised serious doubts about the rule-based, traditional security systems. As a result, with the second generation of XDRs, AI and Machine Learning technologies have been embraced as much as possible in an effort to raise the system's awareness to patterns and malicious intents, through analysing large volumes of past data (datasets) and present data (logs arriving to the system through endpoints, layers, network, etc).

The integration of Artificial Intelligence and Machine Learning into cybersecurity has driven a transformational shift, significantly enhancing the ability to detect, respond to and mitigate complex cyber threats [8], with AI becoming a cornerstone in advancing cyber security, offering innovative solutions to detect and mitigate cyber threats [9].

AI and ML are recognized as effective tools that enhance cybersecurity systems and allow for real-time analytics and decision-making while supporting applications such as intrusion detection, malware detection and network security [10]. AI gives systems the ability to think

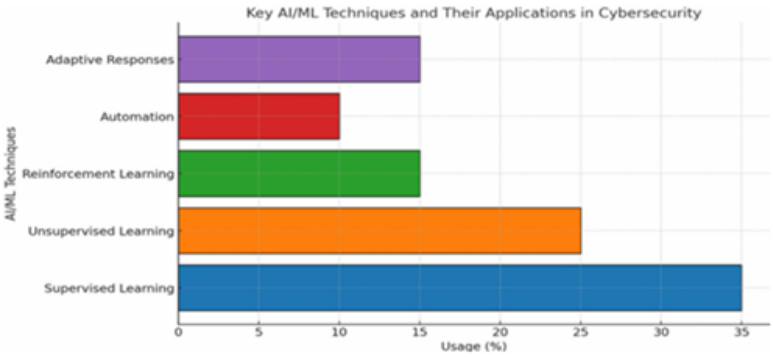

Figure 2.4: The key AI and ML techniques applied in cybersecurity [19]

and reason just like humans, incorporating pattern-recognition and problem-solving. In the vast world of machine learning techniques used in cybersecurity, three key approaches stand out: supervised, unsupervised and reinforcement learning [11].

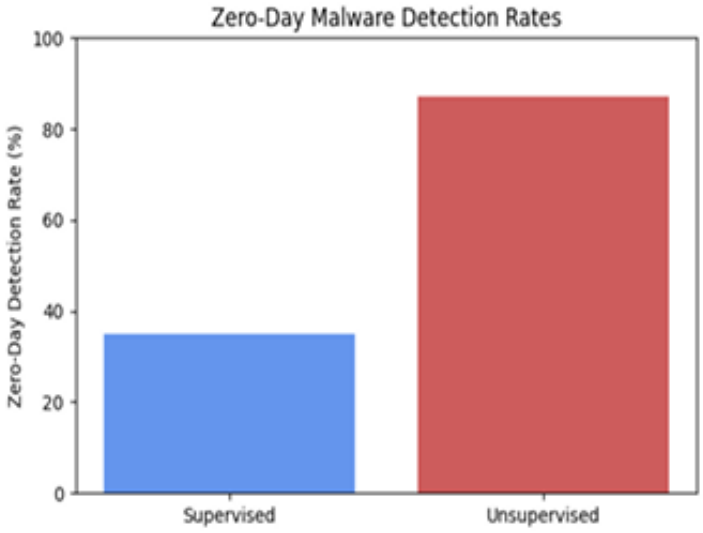

Figure 2.5: Zero-Day Detection Rate by Approach [11]

2.4.1 Learning Methods Overview

In concrete terms, Supervised Learning is designed to work with the use of labels, giving the XDR system a clear path of how to detect and correlate between irregular or suspicious patterns. Unsupervised Learning is used without specific labels, giving the system a different "perspective" of anomalies or unusual patterns. Lastly, reinforcement learning is an approach where agents essentially learn how to spot and contain malware by engaging in trial and error within simulated environments. The learning agent decides whether to classify, quarantine or allow a sample, adjusting its strategy based on the outcomes [12].

Supervised learning shines in controlled settings, unsupervised learning in spotting zero-day attacks, while reinforcement learning shines in adapting to ever-changing, adversarial situations [12].

This gives the system the possibility to handle any incoming situation and make sure to properly mitigate an incoming issue. One of the most used features of security systems is Anomaly Detection, focusing on identifying deviations from normal patterns (normal system behaviour), allowing organizations to detect potentially malicious activities before dealing any significant damage to the core structure. These detection sub-systems are trained through the use of large datasets, one of which is the CICIDS2017 dataset, developed by the Canadian Institute for Cybersecurity [13].

To reiterate, these advancements in AI and ML are good, but also present some issues and challenges when applying them in real-world solutions. First, the issue of diversity between logs and information that systems are trained with. If the data is not diverse enough, the system will not be able to see through unique or zero-day attacks. Conversely, if the dataset is more diverse than usual, the recognition of daily suspicious patterns becomes hard to see and prolongs automated responses for small anomalies.

It is also worth noting that AI allows new vulnerabilities in cybersecurity. Criminals, adversaries, and malicious actors use AI and other technologies to circumvent detection systems and automate attacks with minimal human intervention. The dual-use nature of AI raises major risks in the areas of cybersecurity, privacy, and public trust [14].

Threat Category	Attack Vectors / Tools	Real-World Incidents	Defensive Strategies
Deepfakes & Synthetic Media	GANs, Voice Cloning, Face Swaps, Text-to-Video	Political deepfakes in Canada, AI-generated scams with celebrity voices [1], [2], [3]	XAI frameworks (n-gram analysis), wavelet-based detection, human-in-the-loop review, regulation (Digital India Act)
Adversarial AI Attacks	FGSM, PGD, C&W, CleverHans, ART Toolkit, Poisoning	Adversarial image classification errors, model evasion in CAVs [8], [14], [15]	Adversarial training, Defensive distillation, Gradient masking, Certified robustness
Automated Malware Generation	FraudGPT, WormGPT, Polymorphic Engines, Obfuscators	AIIMS Ransomware (2022), WannaCry, BlackMamba AI malware [20], [22], [24]	EDR/XDR systems, AI-based behavior monitoring, automated incident response, UEBA
AI-Powered Phishing & Scams	LLM-generated phishing, Deepfake voices, Chatbots	Pig Butchering scams, CEO voice fraud via AI [27], [28], [29]	Weighted linguistic pattern detection, biometric authentication, scam detection models
Social Engineering Automation	AI Chatbots, Synthetic Identities, Face + Voice Deepfakes	$25M Hong Kong executive fraud via Zoom deepfake, AI-driven dating scams [27]	User education, deception detection software, image/audio verification, digital literacy

Figure 2.6: Taxonomy of AI-Driven Cybersecurity Threats and Corresponding Defensive Strategies [14]

2.5 Log Analysis and Automated Response Systems

The analysis of logs for anomalies is an important research topic with practical importance in the field of failure analysis and security threat detection. Logs are generated by software-driven applications running on systems or devices to provide information to aid software developers and system engineers with their analysis of system state and condition [15]. Logs are required to be generated by any software in order for the security systems to monitor ongoing processes

and maintain a level of security, by detecting any unusual patterns or investigating incidents based on suspicious activity. Due to the increasing volume and complexity of this generated data, effective log management and analysis is being prioritized for development, as it is one of the most effective ways to improve threat detection.

2.5.1 Currently Well-Established Security Systems & Innovations

Currently, the most utilized system for gathering logs and processing extracted information are SIEMs, with notable mentions such as Microsoft Sentinel and Splunk. These systems collect, aggregate and correlate log data, empowering network defenders to monitor activity and uncover advanced cyberthreats [16]. The correlation of events from multiple platforms helps detect sequences of related events and highlight abnormalities that may not be immediately visible to a human when manually checking the logs [17].

Setting SIEMs aside, most organizations that require cybersecurity services rely heavily on automatic response systems, such as SOAR (Security Orchestration, Automation, Response). These systems not only automate their responses based on the detected pattern of abnormalities, but also co-operate with other security tools to implement a specific course of action, coordinated throughout multiple tools, against any suspicious activity that is detected [16]. The responses can vary from normal ones, such as alert generation or blocking suspicious IP addresses, to complex ones, such as isolating a compromised device or terminating a malicious process.

However, these security systems still work based on predefined static rules, which limit their ability to evolve and handle complex unknown threats, with humans having to check every alert and decide whether the appropriate response is being used by the system.

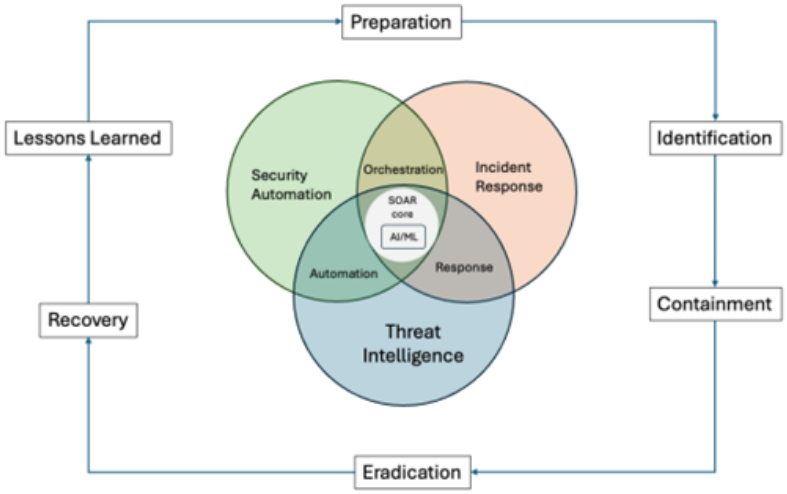

Figure 2.7: Integration of AI/ML into SOAR platforms

These limitations, along with the evolving cycle that threats have gone through—from virus propagation and phishing, to large-scale attacks that blend stealth, automation and

persistence [18]—have shifted the cybersecurity field towards the AI approach, where more intelligent systems can be developed to effectively detect, respond to, and mitigate even complex cyber-attacks.

2.6 Autonomous AI and Research Gap

To reinforce what has been established in the previous sections, the evolution of cyber threats has increased the demand for protection against such threats, but with a difference from the traditional approach: more automation and intelligent adaptation to cyber threats (both known ones and zero-day) and minimal human intervention. While traditional security systems are momentarily still a secure choice for any organization or state infrastructure, with these threats advancing and with the way that organizations are scaling towards much bigger infrastructures, human intervention will become much harder and inefficient when faced with a multi-scale attack.

2.6.1 Autonomous AI Systems & Agentic AI Security Solutions — an Overview

Autonomous AI systems are designed for the simulation of decision-making processes, by firstly analysing and processing all the data given to them, determining visible patterns, and then deciding on the risks of different actions that they initiate. In security solutions, these sub-systems can work as a combination of different security tools, in order to detect and determine whether an attack is coming, and then execute response actions with minimal to no human intervention. One of the key advantages of these AI/ML systems is their ability to adapt to incoming threats, then develop specific patterns of response actions to deal with them, therefore evolving alongside the threats they are fighting against [19].

Figure 2.8: Adaptive defence system for cybersecurity [20]

The picture above gives an illustration of how an autonomous AI system currently works, divided into components for a better visual representation. First, we have the data capturing tools, which are usually watching and monitoring all the available events in the network for

logs, such as Winpcap (Windows) and Libpcap (Linux) [20]. After that, data preprocessing tools consider all these events, and try to separate the ones that they have already learned from or mastered, filtering them out, and leaving only the ones that are unknown. The next process, of feature extraction, as its name suggests, extracts features that make this event unique and suspicious, which then pass it to analysis engines, in order to form a plan of response. After responding, the next layer of security activates, the Intrusion Detection System (IDS) [20].

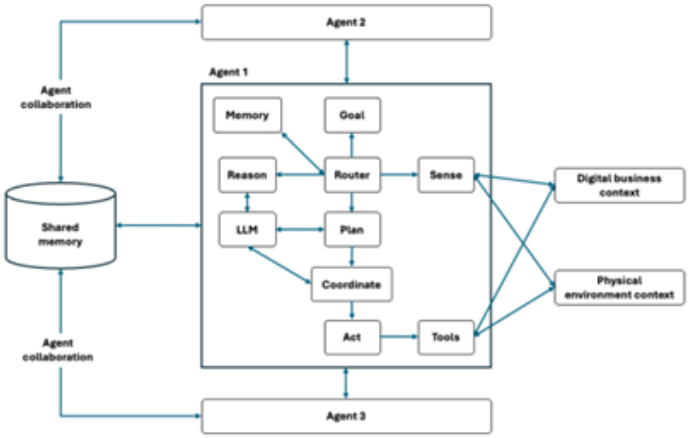

Figure 2.9: Agentic AI anatomy [23]

Another example of an innovative technology that integrates AI and LLMs in cybersecurity is Agentic AI. Nowadays, most large language models serve a key role in data preprocessing, and response generation in regard to the question or problem posed. They are able to process very large amounts of data, recognize patterns, learn from them and identify any patterns that can result in malicious activity [22]. Moreover, its capability of reading rules and guidelines, along with detailed analysis about certain attacks, helps analysts create a more structured plan of response, even without executing that response itself [22].

2.6.2 Current Problems of AI & ML Security Solution Approach

The aim of this thesis is not only to support the use of Artificial Intelligence in cybersecurity, but also to show the gaps that are currently present, and how the integration of Cybersecurity, AI and Machine Learning will help fill them. One challenge that we're faced with in this approach is the low amount of XDR-focused simulation systems. Existing studies focus on sole features, such as Machine Learning patterns for detecting anomalies, or rule-based automation using AI, therefore not creating a single instance application which integrates both these functions and gives more functionality in one tool [21].

Rule-based systems can also be integrated with other security systems, such as intrusion detection systems and SIEM systems [21], thus making a more complete security solution that covers all the required functionalities.

Another issue is the problem of trust in the accuracy of threat detection and response decisions.

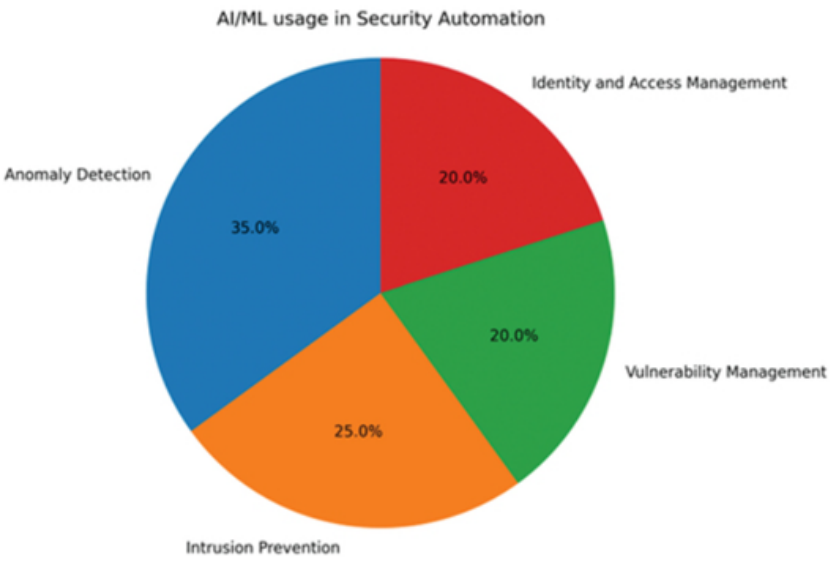

Figure 2.10: AI/ML in Security Automation [21]

This is a persisting issue, present even in current deployed security solutions, where these systems lack the ability to adapt to dynamically evolving threats [22], and therefore cannot be fully autonomous.

2.7 Ethical and Operational Challenges of AI-Driven Cybersecurity Systems

Now that we have established the importance of integrating AI in cybersecurity, with two of the most important reasons being the speed with which threats are evolving day by day and the improvements that AI has shown for the effectiveness and efficiency of threat detection and response [24], it is also worth noting that this integration should occur by taking into account several ethical and operational issues. The need for more intelligent systems should also bring the need for concerns over the reliability and transparency that these proposed security solutions offer, in comparison with conventional systems.

Figure 2.11: ChatGPT User Interface Disclaimer

2.7.1 Reliability and Accuracy of AI Decisions

One of the main concerns surrounding the current state of AI is the reliability that it offers to its users. From simple things like searching for information, asking a question or posing a problem, to complex situations such as detecting suspicious patterns and anomalies, there is always a label that any AI software uses to inform the entity using it that AI results may not be correct, and need further verification. Machine Learning models depend heavily on

training data, pre-processed by LLMs, and issues such as data quality, bias, and incomplete or poisoned datasets (an attack technique used in Cybersecurity) can negatively affect detection accuracy and system performance [21].

For a normal user, this can only be a minor inconvenience, as the sources of information are infinite nowadays. However, for a cybersecurity system facing a cyber-attack, one wrong mistake can compromise entire systems or even the whole infrastructure of the organization, causing full-service disruption. Therefore, creating a set of standards and best practices to ensure high detection accuracy, and to minimize any decision errors, is a main priority.

2.7.2 Transparency and Explainability

Another critical issue that needs to be addressed is the transparency of a model's reasoning for its actions and responses. Most AI large language models work in the form of Black Box systems, which means that the process they go through to evaluate the best course of action is not easily interpretable [25], making it harder for humans to understand how the model arrived at a certain point, and further validate its response. An analyst must make sure to understand why an alert was triggered, how the pattern was discovered, and if the action set as a response is correct or not. This is also a very important feature of the envisioned fully autonomous security systems, as the decisions of these systems should be fully explainable to ensure trust and accountability [25].

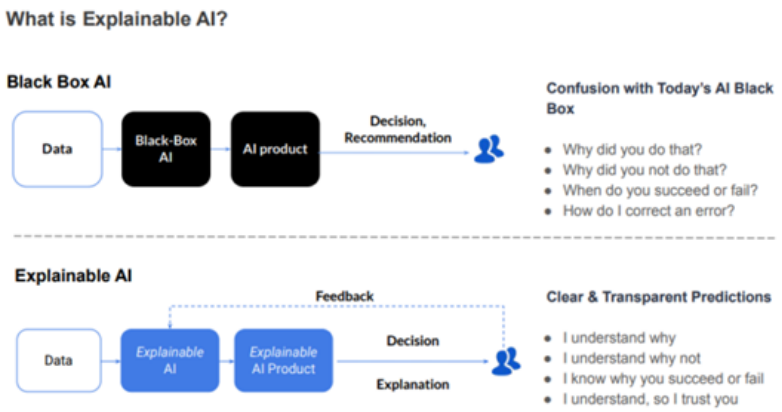

Figure 2.12: Explainable AI vs. Blackbox AI [26]

2.7.3 Security and Privacy Concerns (CIA Triad)

AI cybersecurity systems rely heavily on LLMs that process heavy amounts of data, be it data from external sources (e.g. CICIDS 2017/2018) or internal sources such as the logs generated by the events of the organisation infrastructure. Part of these internal logs might contain sensitive information that needs to be handled carefully. The CIA triad (Confidentiality, Integrity and Availability) forms the cornerstone of information security practice and theory,

pointing out the most important objectives that should be followed by these systems to guarantee a safe environment.

Furthermore, AI models as mentioned before might also be attacked through the datasets that they use to learn from, using data poisoning [27], that involves either of the following actions:

- Overfitting the data with the same type of information, thus making the model not adaptable to new threats.
- Incorrect logs that help create fake patterns that will be useless for detection or response action decision.

Another present issue is the source of information from where these events and logs are generated. Second-generation XDR platforms quite often rely on different sources of information, making it important to evaluate the validity of the information being used, so as to not increase exposure to threats.

Figure 2.13: CIA Triad [28]

2.7.4 Importance for Next-Generation Security Systems

Even though presented with the current challenges, the field of AI, especially Agentic AI, is evolving rapidly, making it possible to think that all these issues can be resolved. The envisioned intelligent systems will provide a much better experience than the current ones, with more autonomy in detection and decision-making. AI-driven security systems will cover multi-platform security functionalities (detection, adaptation and automated response actions), forming the foundation for fully automated and proactive system architectures [29].

Figure 2.14: Incident Response Lifecycle [29]

2.8 XDR Systems and Compliance with Cybersecurity Frameworks

XDR + integrated AI systems are considered to be the next destination that cybersecurity is heading to, considering how threats are evolving and organizations are continuously scaling their infrastructure. XDR solutions really do offer compelling benefits; however, companies that are already implementing or will implement them face significant challenges [30]. These issues can be mitigated by implementing these solutions in compliance with specific standardized cybersecurity frameworks.

The integration between these systems and cybersecurity frameworks helps identify threats earlier by using well-known techniques, as well as creating a standardized set of procedures that should be followed when responding to an incident. This section will further explain how XDR solutions can work together with some of the most relevant frameworks: MITRE ATT&CK, NIST and ISO/IEC 27001.

2.8.1 MITRE ATT&CK Integration in XDR Systems

The MITRE ATT&CK is one of the most widely used open-source frameworks in cybersecurity solutions. It is a globally accessible knowledge base, containing documentation about real-world threats, focusing on three key aspects of attacks: tactics, techniques and procedures (TTPs) [31].

This framework maps detected suspicious activities, reported in publicly accessed databases, to known attacker behaviour, helping organizations detect and neutralize threats much faster. Not only that, but XDR systems can ingest information from MITRE ATT&CK, and use it to triage alerts, enrich cyber threat intelligence from different sources and ultimately test the effectiveness and accuracy of the incident response actions that it implements autonomously [32].

This framework also helps XDR systems to better understand cross-platform threats, such as multi-stage intrusions that usually happen in broader networks, where the XDR is capable of detecting suspicious patterns throughout all connected platforms.

It is also worth noting that the TTPs, characteristic of MITRE ATT&CK and the way their information is organized, make the use of this data for training AI models much more feasible.

Figure 2.15: The MITRE ATT&CK Framework [33]

2.8.2 NIST Cybersecurity Framework and XDR

The NIST Cybersecurity Framework (CSF) is a unified framework that provides a structured approach for managing a threat in a high-risk situation [34] using five core steps:

1. Identify
2. Protect
3. Detect
4. Respond
5. Recover

These steps are a functional dependency for XDR and AI security systems, which in turn provide much better detection and visibility towards incoming threats, enabling a fast incident response time. Using events from multiple points of the network and unifying data from these endpoints, XDR systems enable real-time threat detection and automated response actions.

Furthermore, the NIST Cybersecurity Framework unifies the language for risk management, enabling organizations to align security operations with business and compliance requirements [34].

Figure 2.16: NIST Framework [34]

2.8.3 ISO/IEC 27001 and XDR Systems

ISO/IEC 27001 is a cybersecurity standard recognized internationally for Information Security Management Systems (ISMS). It provides organizations with a structured framework of rules and procedures for managing sensitive data and guaranteeing its safety and security.

This standard uses a risk-based approach, where organisations continuously monitor security risks through set policies and procedures, meaning that ISO 27001 provides a comprehensive and structured framework for protecting information assets and reducing the likelihood of data breaches and cyber incidents [35].

One important security measure stated by this standard is to always set up event logging and monitoring to record system activities and provide help in threat detection and response. XDR systems align closely with this requirement, as they usually collect information of events from multiple sources, giving analysts a centralized logging system.

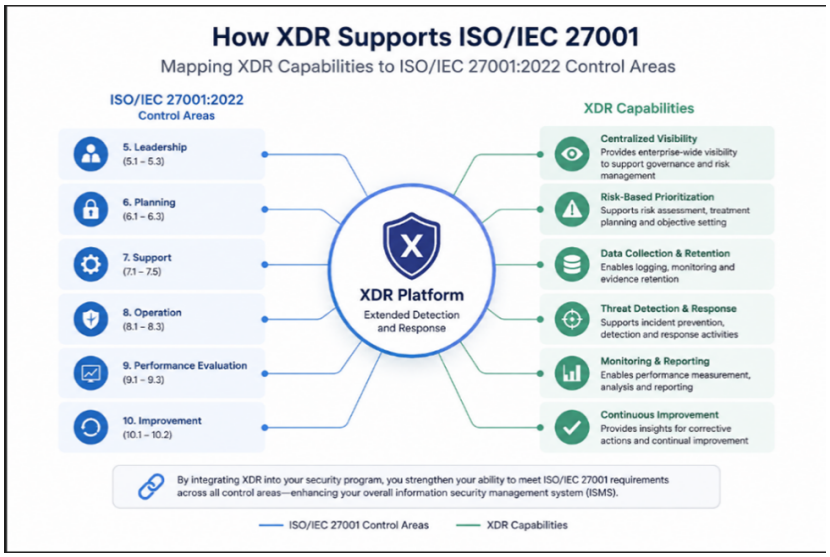

Figure 2.17: Conceptual relationship between ISO/IEC 27001 and XDR systems (generated using AI tools)

2.9 Compliance Challenges and Operational Limitations in XDR Systems

As mentioned before, XDR systems are the next line of innovation. Along with the enhancements from AI and ML, as well as the centralization of data, they will be a huge service requested by all organizations. However, much like the traditional systems currently used in most cybersecurity solutions (SIEMs, EDRs, SOAR, etc.), they must fully comply with well-known frameworks and best practices. This might be a challenge, considering that AI tends to be inconsistent with decision-making, where the algorithms that give AI the ability of learning and operating autonomously are not yet fine-tuned to fit the steep requirements that the cybersecurity field has.

2.9.1 Transparency and Auditability Issues

As mentioned before, AI-enhanced XDR systems work like a black box, where the analysts have difficulties interpreting why the system responds a specific way to a specific incident or occurrence. The decision process of these tools is impossible to interpret [36].

These issues for auditability and transparency cause big problems, as cybersecurity analysts need to know and understand why a particular event is deemed malicious, and why a specific incident response action is taken against it, in order to fully comply with regulations [37]. This is also an issue for organisations that might be facing an APT (Advanced Persistent Threat), where the threat is not easily found, making it hard for the analysts to follow up with understanding why the AI and ML-enhanced XDR system decided to mark a series of activities as suspicious.

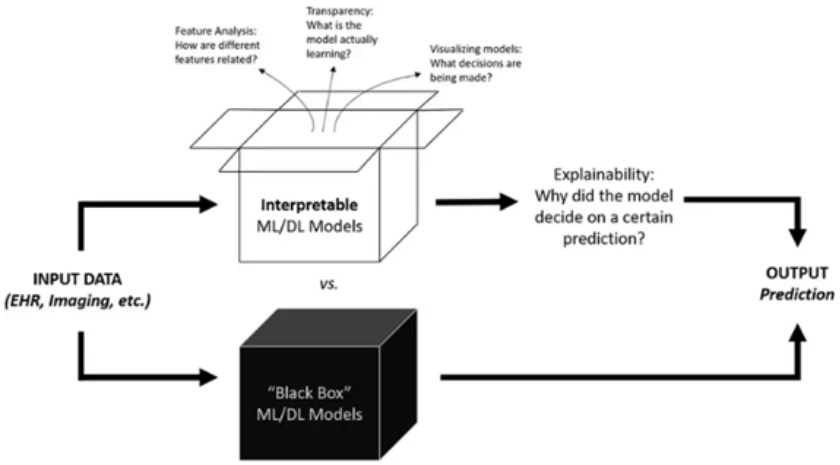

Figure 2.18: Black Box Concept vs. Explainable AI [40]

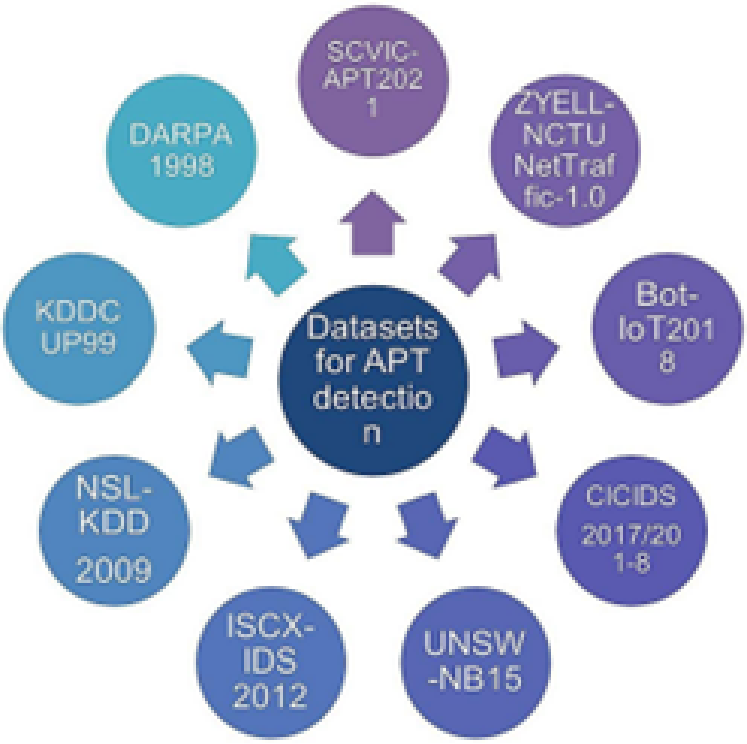

Figure 2.19: Datasets used for APT detection [37]

2.9.2 Operational Complexity and Integration

However, even with the use of AI and strong ML algorithms, analysts can never be one hundred percent sure that everything that the system deems appropriate is actually appropriate. This begs the question: with centralized data coming from multiple points of an organization's infrastructure, how can an administrator handle all this information? They could be rapidly overwhelmed by the data and have difficulty correlating the events [37].

Not only that, but the policies enforced should always be firm and consistent, and the response plan should be standardized, making it even harder for a human analyst to keep up. These issues become even more problematic when having to deal with largely scaled security solutions that integrate multiple tools together.

2.9.3 Limitations in Automated Compliance Enforcement

Although the above-mentioned frameworks, such as MITRE ATT&CK and NIST CSF provide structured and clear guidance on how to operate safely, the XDR system does not automatically comply with them.

The MITRE ATT&CK provides a series of TTPs for well-known cyber-attacks, but that does not mean that an XDR system should have it as a predefined rule to function a certain way for both detection and response, thus making MITRE ATT&CK a reference, and not a main source of automated compliance.

The same thing can be said about NIST CSF: even though it gives security providers a clear way of defending an organization's system, it does not mean that the steps will be followed rigorously using enforced automation, but only through complex organization and planning [39]. Therefore, an XDR system, even the innovative ones including LLMs and powerful ML algorithms, still requires manual intervention in some cases.

2.10 Summary & Review of Gaps and Future Research Scopes

This chapter provided a review of every modern technology related to cybersecurity systems, especially focusing on threat and intrusion detection as well as incident response, which are the two core functionalities of XDR systems, while also provisioning the capabilities and innovations of these systems incorporated with Large Language Models (LLMs).

It started with traditional and conventional systems, such as SIEMs and EDRs, focusing on their way of operation as well as their positive impact in providing security, but also covered their current limitations and their disability to evolve along the threats that are being developed every day.

The next part covered one of the most important concepts of what the cybersecurity field is trying to achieve: systems that use LLMs, incorporated with Agentic AI and powerful

machine learning algorithms, to create a centralized dashboard for analysts to easily maintain a whole infrastructure.

Lastly, the chapter also covered the relationship between these XDR systems and cybersecurity frameworks, highlighting the importance of the protocol and guidelines that these systems should follow on every scenario, as well as learn and understand how attackers use tactics and procedures to infiltrate the system and how to follow a specific set of actions in order to neutralize large-scale threats.

Chapter 3 – SYSTEM DESIGN AND ARCHITECTURE

This chapter contains insights about the design, development and simulation of the proposed AI-driven XDR security system. The main points include the overall structure of the solution, how the components work with each other, and describe the tools and technologies used for creating it.

The solution's aim is to simulate the behaviour of modern XDR (Extended Detection & Response) systems, by collecting telemetry from more than one source, analysing generated logs from these sources, then identifying any weird behaviour or pattern, to finally automate a response plan. The way this is going to be achieved is by using a centralized system with powerful Machine Learning algorithms.

3.1 System Overview

The system's design can easily be described as an AI-driven XDR security system, that can not only do what an XDR system alone always does—such as collecting data from multiple sources and centralizing the display of these data—but also detect suspicious behaviour over different data layers, correlate this behaviour from one data layer to another, and then respond to this issue autonomously with the appropriate response plan.

All the components of this simulated solution work with each other, all handling different parts of the process (detecting or responding), with a main goal of replicating a real XDR system used in real-world cybersecurity environments. One of the ready-made components that will have to be included is an open-source telemetry system called Wazuh, alongside a trained AI model, capable of detecting abnormal patterns and concluding which response action is best appropriate for the threat.

The above picture gives a visual representation of the layered design, starting from the endpoints that are being monitored, continuing with Wazuh agents grabbing these logs and sending them to the detection engine, where powerful Machine Learning algorithms try to detect abnormalities and identify any possible threats. If there is any suspicious behaviour detected, the decision engine powered by an AI model determines the severity of the event and simulates a response plan. The response plan includes things such as process termination, endpoint isolation and IP blocking.

3.2 Endpoint Monitoring and Log Collection Layer

The first step of the proposed XDR system is endpoint monitoring and log collection. This layer is used for simulating the generation of telemetry, collecting it and then sending it to a

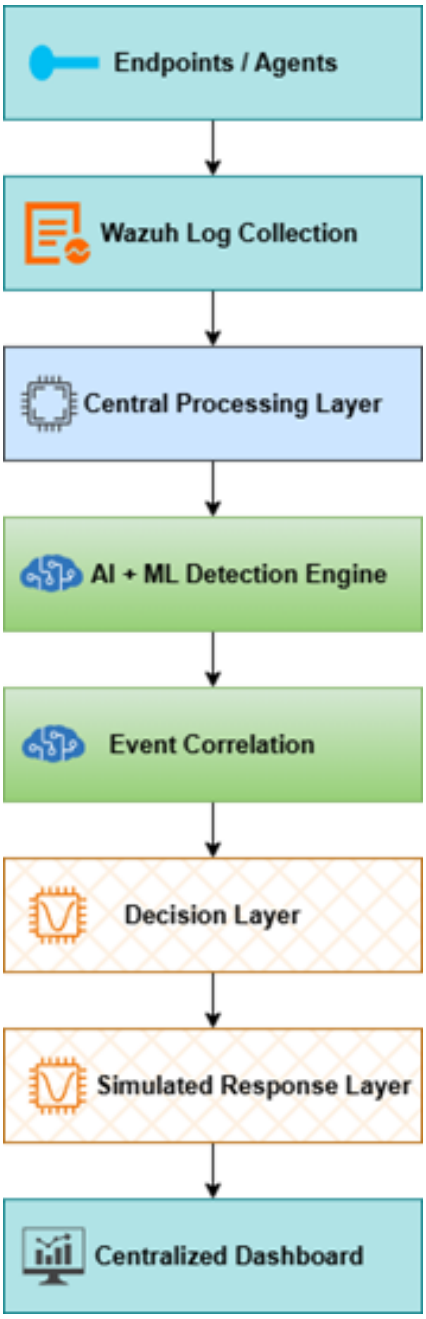

Figure 3.1: XDR Simulated System Design

centralized processing layer.

The main components are the Wazuh agents, which are installed on the monitored endpoints (in this case Virtual Machines). They collect data and logs from events and processes and then send them over to the Wazuh manager for further analysis.

The collected telemetry data can include normal process logging, but also:

- Authentication events (not in normal work hours)
- Failed login attempts (suspicious if the user has tried multiple times)
- Suspicious PowerShell activity (such as the command `whoami`, to understand user access and privileges)
- File modification activity (suspicious if a process tries to modify files that are core to the operating system, or that include batch processes)

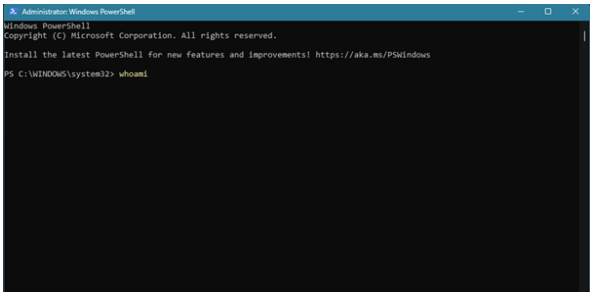

Figure 3.2: `whoami` Command in PowerShell

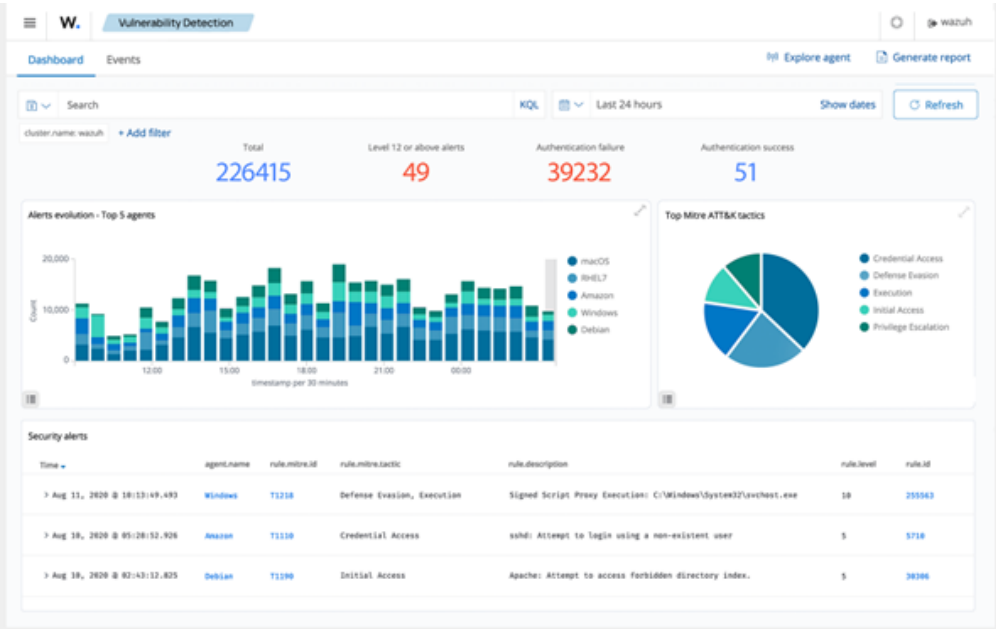

Figure 3.3: Wazuh Central Server (Dashboard), accessed on May 23, 2026

These processes collected by Wazuh agents, after being sent to the main server, are normalized and categorized based on already defined rules of category and severity. This gives the AI

engine more capability to understand whether any of these logs represent an abnormality, or if there is a connection between two or multiple logs that may indicate a possible threat.

3.3 Central Data Processing and Telemetry Management Layer

This section covers the second part of the solution, specifically talking about how the data taken from numerous sources is connected to the AI engine which will actually analyse the telemetry. Anything collected by the Wazuh agents will be sent here, so that the machine learning trained models can analyse and search for patterns or abnormalities through a wide number of generated logs.

All the generated telemetry is collected and processed by Wazuh Manager (centralized server storing all the data), to then be sent through the central processing layer. The most important thing to do at this stage is to create a standard format for all the logs, which will help the models better understand the telemetry. This will be done by normalization and use of data sanitization methods.

The desired end-product will be a prepared dataset that will be further analysed, but there will also be another crucial step: feature extraction & labelling. Machine Learning algorithms are unable to just train themselves on raw telemetry or log data, meaning that specific attributes will have to be labelled as important variables in the detection and decision-making parts. Table 3.1 shows a raw example of how labelling is made.

Table 3.1: Feature Labelling Example

Feature	Description
Failed Login Count	Suspicious number of login attempts
Event Frequency	Similar events with same interval apart
Source IP Address	Activity origin
Process Name	Name of executed process
Endpoint Severity	Determined by the Wazuh Manager
Endpoint Identifier	Finding the source of telemetry

The design choice of having a different layer for data processing and detection logic followed by response is applied to current XDR-based cybersecurity solutions, due to the scalability potential it has for future modifications or additions to the system or to the machine learning algorithms, also making it easier to just add new endpoints.

3.4 Artificial Intelligence Detection Layer

This is the core part of the XDR solution, offering detailed analysis of all the normalized data and telemetry received from the Wazuh Manager Server and any real-time logs, helping identify any discrepancies or abnormalities in a log or a series of logs. Unlike conventional systems, this will rely not only on basic predefined rules, but also on behavioural analysis

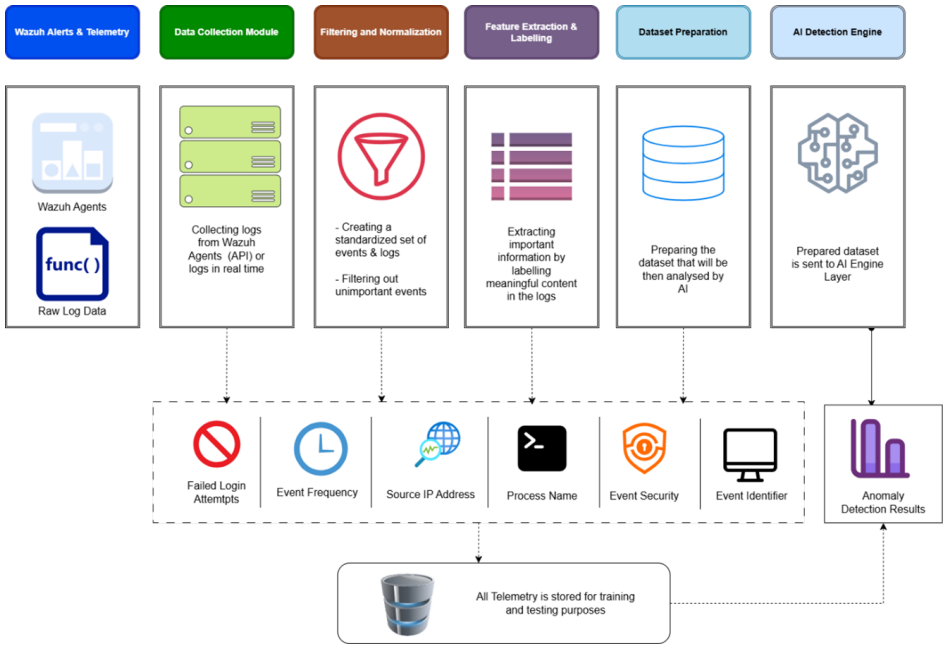

Figure 3.4: Central Data Processing Workflow

where the most innovative part is the detection of threats that don't use a known pattern of attacking.

The evaluation of the normalized telemetry data is done through powerful Machine Learning Algorithms, focused on extracting features such as:

- Frequency of a process being executed
- Network behaviour
- Process execution and resource patterns
- etc.

These features are selected since they determine if a particular event or a thread of executed processes is of risk or not. The detection phase begins with taking all the extracted features, sending them to the Machine Learning model, which will classify all the data using an anomaly score, where the higher the score, the higher the chance of this event or log being an indicator of a cyber threat.

The design with two different layers—where one classifies potential risks and the other correlates them—makes it easier to scale the models or the layers themselves.

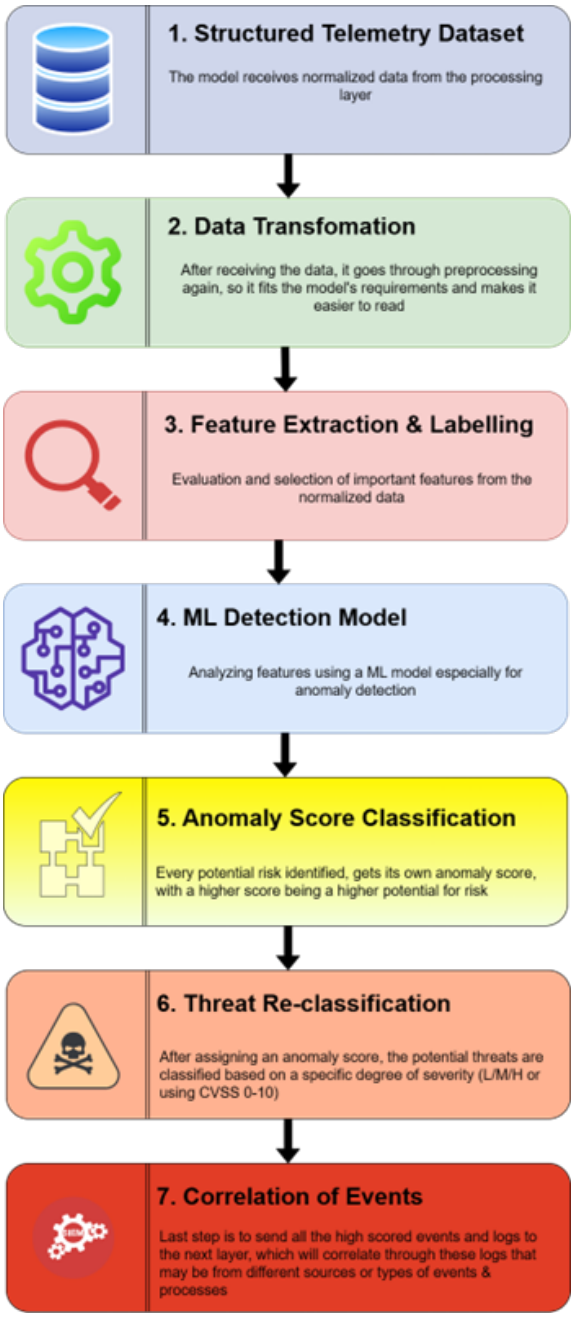

Figure 3.5: AI Detection Layer Workflow

3.5 Event Correlation and Threat Analysis

The correlation layer is right after the detection layer decides on what to classify as potential threats, by filtering the suspicious telemetry data out and sending it to this layer. The correlation logic will evaluate the risks using metrics such as:

- **Threat Scoring:** The higher the score, the higher the chance of a log or event being related to a cyber threat.
- **Attack Pattern Matching:** Known sequences of scaled attacks and TTPs (using MITRE ATT&CK's framework) are used to determine whether a sequence of logs matches a

well-known attack.

- **Confidence Evaluation:** Verifying whether one of the filtered logs can actually pose a threat or is just a false positive.

An important objective of threat correlation is to identify if there is a connection between different logs and tie it to a largely scaled attack or intrusion. However, it should also be able to detect whether the filtered-out logs by the detection layer can be coincidences or false positives. Through continuous learning and training, better data normalization and feature extraction, as well as model improvement, it can help create a stronger and more accurate layer.

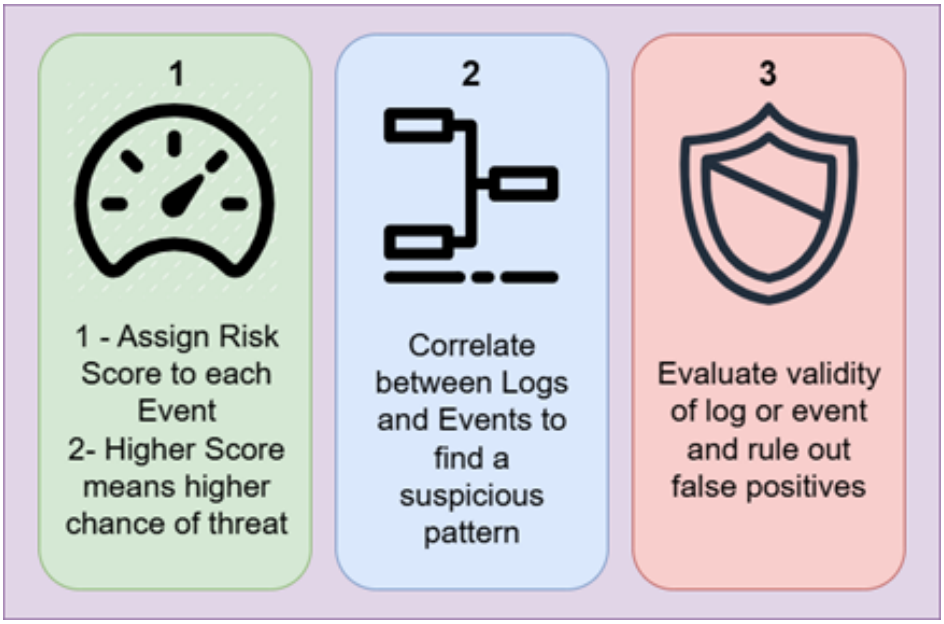

Figure 3.6: Correlation Layer Architecture

3.6 Decision and Threat Mitigation Layer

The Decision layer is responsible for validating the results that the AI model has come up with after suspicious log detection and event correlation. After evaluating the results, it determines the appropriate response action, acting as the rational thinking and decision-making engine, which is required in real-world XDR solutions. This feature must be if not fully, mostly automated, requiring little to no human input.

The automated process is executed based on some performative features, such as:

- **Severity** (based on the score assigned from detection)
- **Confidence** (based on how confident the detection model is that the filtered suspicious log is actually an anomaly)
- **Context of detected incident** (The initialization of the attack and if there is usage of

TTPs)

Based on previously generated data such as anomaly scores, correlated events, threat classification and affected assets, the model continues with assessing the level of risk. Using this, the model then assigns priority and launches a specific response plan, with examples of responses below:

- Security alert generation
- Increased watch and monitor
- Blocking of suspicious external sources and IP addresses
- Termination of suspicious processes
- Isolation of affected assets & endpoints

However, this response action is not directly sent to the affected endpoints, but actually goes through a testing layer called the Response Simulation Layer, where the chosen response action is simulated to prove the effectiveness of the choice made by the model.

Figure 3.7: Decision and Threat Mitigation Layer

3.7 Response Simulation Layer

The Response Simulation Layer is the last component that makes this architecture complete. The response chosen by the AI & ML model needs to be executed in a controlled testing environment to ensure that the response is actually effective against the threat. This will replicate the actual real-world response actions that are executed in any security solution.

The response actions are comprised of actions meant to limit an endpoint's access to the local network, or even total isolation, thus preventing any malware from spreading to other endpoints in the same network, stopping ransomware from encrypting files and so on. It is

also important that every action performed by this simulation is saved and logged.

All the logged data from an incident must be done cleanly, making sure that the logging process is standardized and every generated piece of information can easily be read by a human, or turned into further training data for future models. Important features include:

- Detected Threat
- Severity Level
- Selected Response Action
- Timestamp
- Affected Endpoint/Device
- Response Status

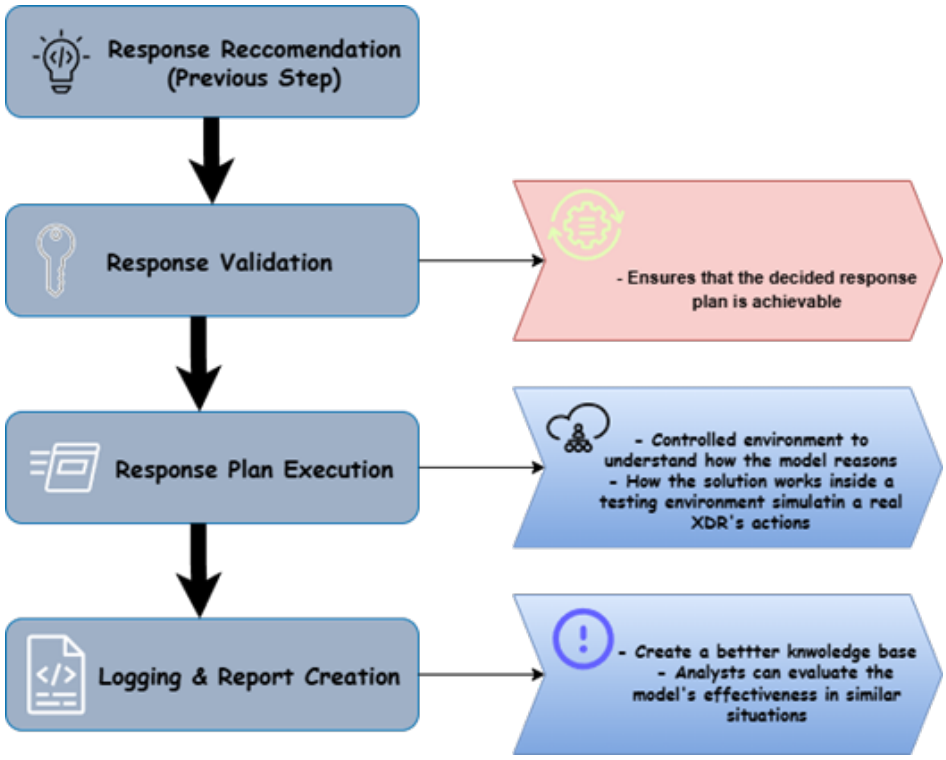

Figure 3.8: Response Plan Execution Workflow

3.8 Dashboard and Monitoring Interface

The dashboard and monitoring interface will serve as the view that the user (analysts) will interact with the proposed solution. Its stack will comprise React & TypeScript, simulated by real-world XDR solutions, that will help show current logs being sent from the different endpoints in a user-friendly structure.

Besides the data, this dashboard will also preview any incoming security events or active alerts that are generated at the moment when data is going through the detection and correlation

layer, as well as the results of AI analysis done on the backend. It will also contain a tab where there will be a chronological order of the issues, giving analysts a detailed view of how the model processed the information and gave back its response.

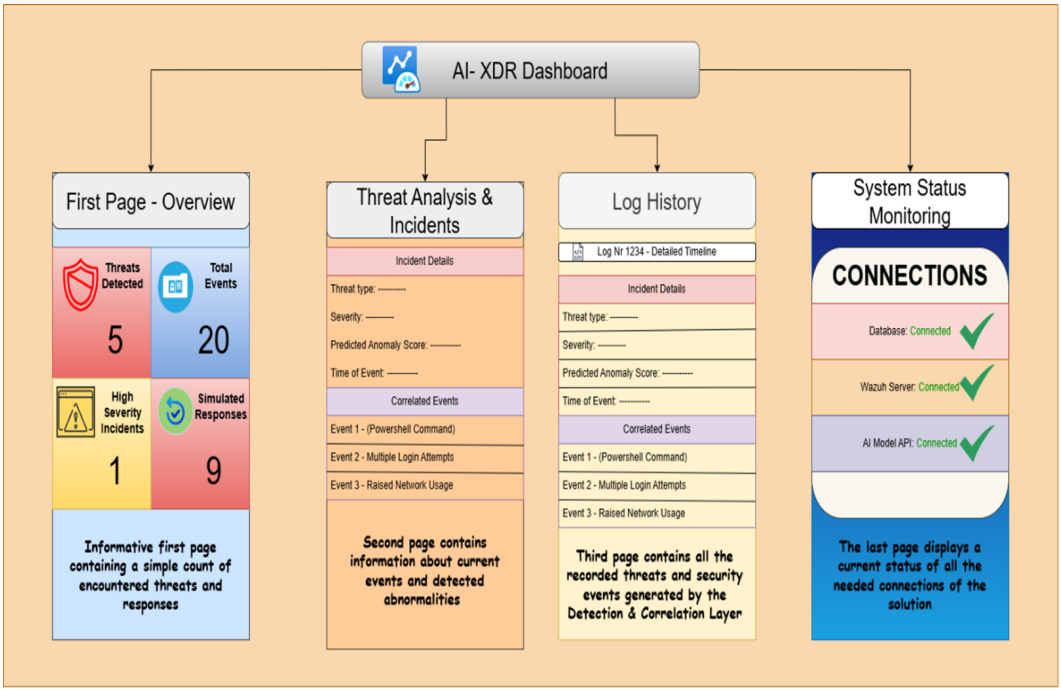

Figure 3.9: Dashboard Pages

3.9 System Modelling: UML Diagrams

The design and implementation of the proposed solution require not only functional explanation and overview, but also a clear correlation between the interactions of users (analysts) and the system itself. The selected diagrams that are able to fully explain how the system works and how users interact with it are:

- Use Case Diagram
- Sequence Diagram
- Component Diagram

3.9.1 Use Case Diagram

The Use Case Diagram is used to illustrate the connection and interaction between system users and the XDR solution. It helps us understand who are the types of users that can interact with the system, and what kind of functionality and privilege levels each group of users has. It also shows how the system actions are connected to each other and can be executed by the users themselves.

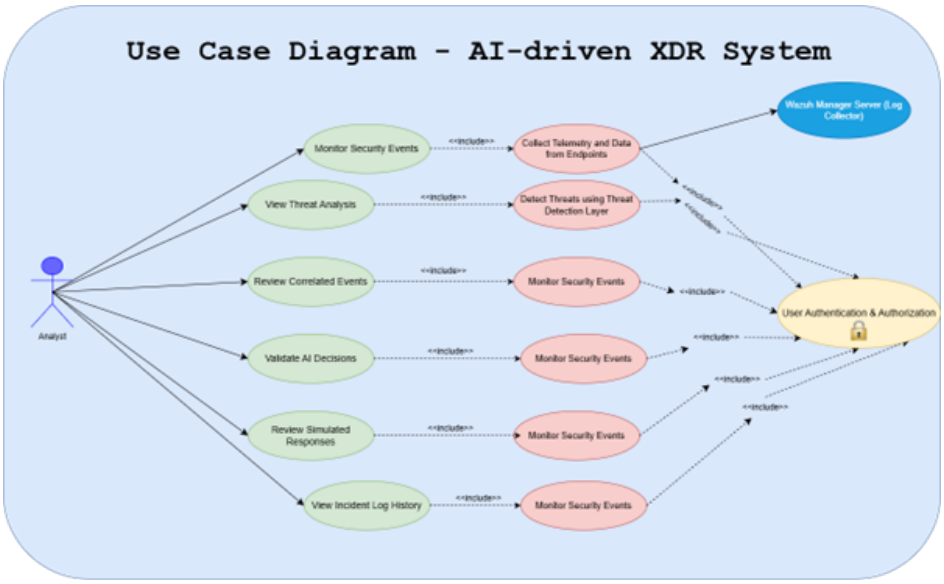

Figure 3.10: Use Case Diagram of AI-driven XDR System

3.9.2 Sequence Diagram

The Sequence Diagram will illustrate not only the interaction between users and components of the system, but will also give a clear timeline of the execution of each process that the system has, in a chronological manner. The process will begin from the endpoints that generate logs and event alerts, which are then collected by Wazuh agents and sent for further inspection. After the data received is normalized and standardized, it goes through three different layers and displays the result on the dashboard.

The components that will be part of the diagram are:

- Monitored Endpoint
- Wazuh Agents
- Data Processing Layer
- AI Detection Layer
- Correlation Layer
- Decision-Making Engine
- Response Simulation Layer
- Dashboard

3.9.3 Component Diagram

The Component Diagram gives an overview of the structural organization of the proposed XDR solution. Different from the previous diagram that explains the timeline of the interaction between the user and the components, this diagram provides the internal architecture, including

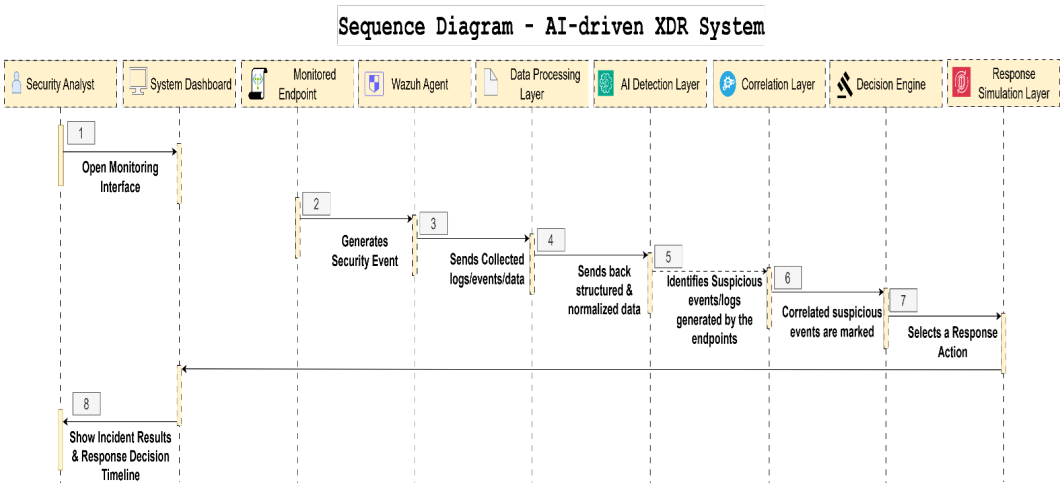

Figure 3.11: Sequence Diagram for XDR solution

the data processing module, AI detection and correlation layers, decision engine and the user dashboard. It also shows the fact that all these components work autonomously, giving the option to modify or change parts of it without worrying about the whole architecture.

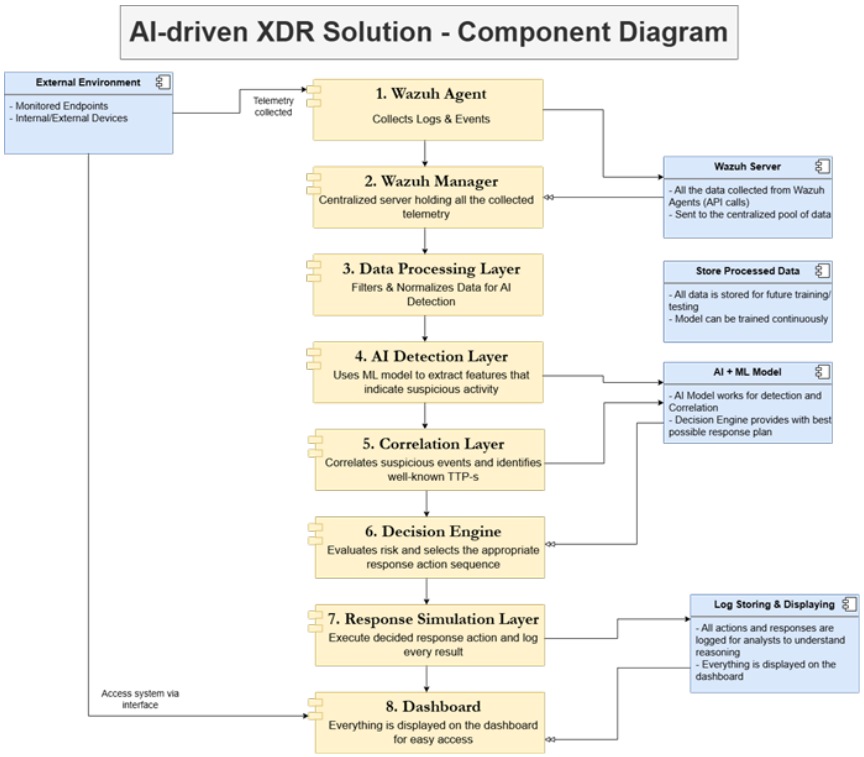

Figure 3.12: XDR Solution Component Diagram

Chapter 4 – SYSTEM IMPLEMENTATION

This chapter covers the usage of the whole architecture laid out in the previous chapter. It is comprised of all the technologies used for model training and testing, as well as the backend and frontend of the XDR simulation system. Every section will explain the implementation of a specific layer or component, including snapshots of the code and the deployed solution.

4.1 Hardware Environment

The AI-driven XDR system was developed and implemented using a Windows 11 workstation, in which was also installed and initialized a virtualized Ubuntu Server, where the Wazuh Manager was deployed.

The host Windows machine was responsible for software development, system dashboard creation as well as endpoint simulation (the log collection mechanism takes logs from both the virtualized Ubuntu Server and the Windows Machine). The virtualized server is run through the use of Oracle VirtualBox. The combination of two different sources for logs not only simulates a real-world implementation of an XDR system but also helps simulate how the correlation of different events from different endpoints may come from the same scaled attack.

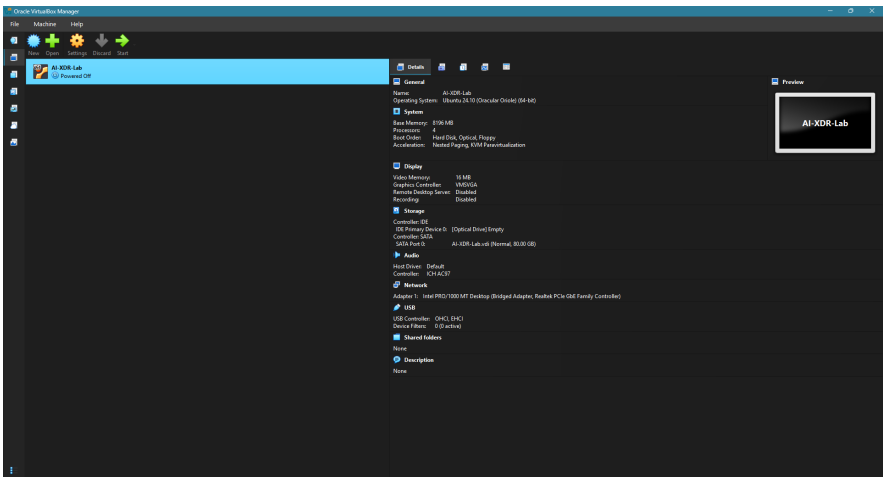

Figure 4.1: Virtual Box Instance

4.2 Software Environment

The implemented system utilizes open-source technologies, specifically listed in Table 4.1 below.

Table 4.1: Software Environment Used in the Implementation

Technology / Framework	**Purpose**
Python, Flask, SQLite	Machine learning model training, anomaly detection, backend API development, and data storage.
React JS, TypeScript, Node.js	Development of the monitoring dashboard and user interface components.
Oracle VirtualBox	Deployment and hosting of the Ubuntu-based server environment used for the simulation.
Visual Studio Code, Git	Source code development, version control, and project management.
Wazuh, Sysmon	Collection, monitoring, and analysis of endpoint and system telemetry data.

Python was used to build the backend service responsible for log processing, AI-driven anomaly detection, threat scoring, correlation and response action simulation. Flask, a lightweight REST API framework, provided a layer of communication between the web-based dashboard and the backend.

SQLite was used to store the logs after processing and standardization, in order to keep them as memory references, simulating a real-world XDR system where logs are stored indefinitely so that analysts can review and try to understand and detect patterns or anomalies.

On the Windows machine, the telemetry was generated using Sysmon, a Windows system service used to generate logs that capture more information than normal events, which were then passed to Wazuh, and after processing and AI anomaly detection, passed to the SQLite database.

For the web-based dashboard, the development framework is comprised of Node JS, React JS and TypeScript. These technologies are used in many modern front-end solutions, making them a perfect fit for the design requirements.

Lastly, Visual Studio Code was used as the development environment, since it offers support for many languages and frameworks adapting to both backend and frontend technologies, along with Git which was used for versioning and source code management.

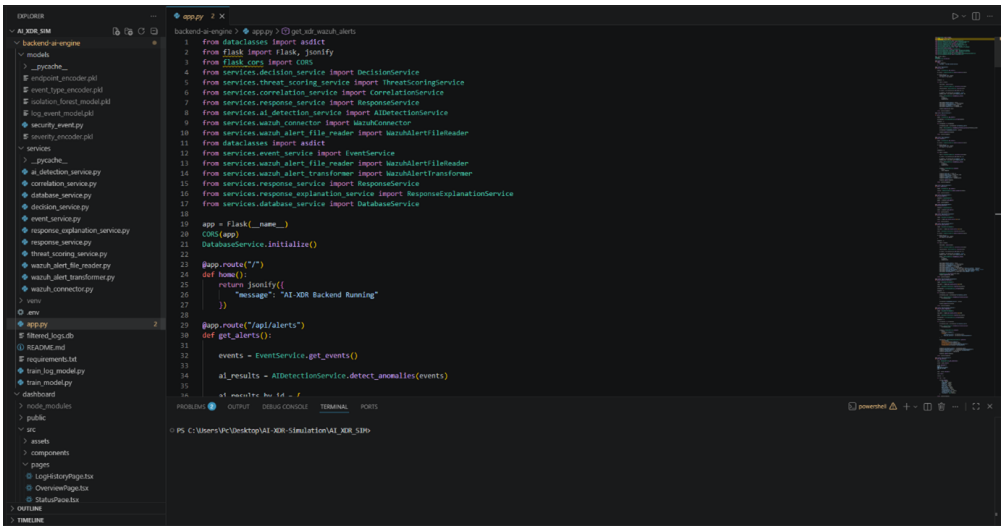

Figure 4.2: Visual Studio Code Dev Environment

4.3 System Architecture Implementation

Following the before-mentioned architecture of the proposed XDR solution in Chapter 3, this section covers the implementation of the architecture with a component-based approach, separating the different layers that data goes through for processing, anomaly detection, AI, correlation and response decision engines, finishing up with the database layer.

The first step of the implemented workflow is the Wazuh SIEM, which is an open-source cybersecurity solution that can be used to manage a number of endpoints' logs and events by centralizing the log collection server, therefore paving the way for anomaly detection between different events and logs.

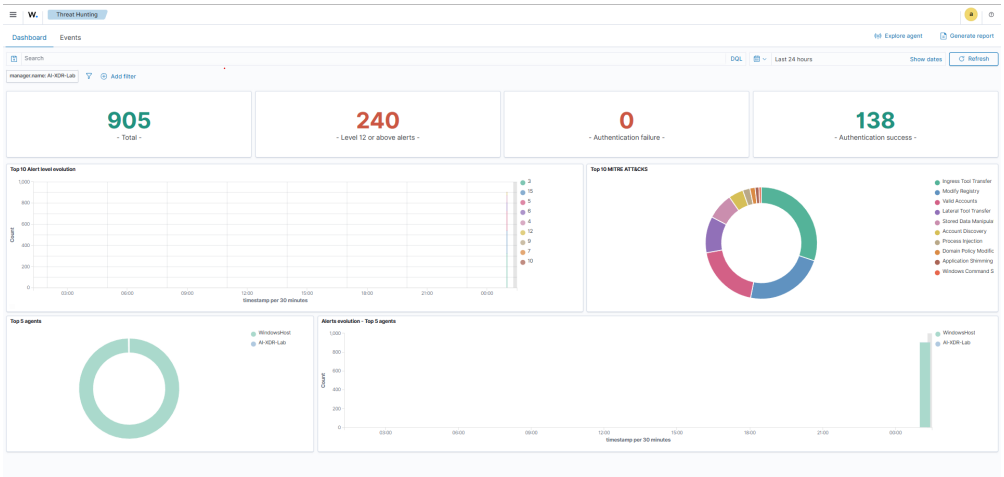

Figure 4.3: Wazuh Threat Hunting

After the logs are collected, the next step is to normalize and standardize them, especially logs generated by Sysmon as they tend to be more detailed and are generated for every step of an

event or process running, so that the trained AI model better understands the data being fed to it. Once normalized, the data goes through the AI Anomaly Detection layer, where each log is evaluated and given a threat score, which gives the XDR system the ability to classify and prioritize specific threats based on impact and risk. After detection, the correlation layer checks the logs to find similarities, be it the same time interval or a nearly identical signature. It also tries to pair it with current MITRE ATT&CK TTP patterns, in order to give more detailed context to the analysts. After correlation, the response simulation layer is set in motion, where the response is simulated based on factors such as threat score and risk/impact, with responses varying from continuous monitoring to process termination or endpoint isolation.

All the logs are stored in SQLite, a lightweight database service provided in Python with a limit of 25 TB of storage, that will store all the logs for analysts to review and understand the decision-making of the AI-driven detection. The dashboard is the last part of this component-based solution, giving the analyst a good view of the data with a user-friendly interface, communicating with the backend using an API service built with Flask.

4.4 Log Collection, Sysmon & Database Service

Wazuh is an integrated XDR and SIEM system, capable of not only threat collection and processing but also features of cybersecurity solutions such as threat hunting, risk assessment and endpoint monitoring and management. Due to it being open-source and free to use, it was an overall optimal choice, since the only functionality taken from this system was the log collection and centralization.

As mentioned in the previous sections, another tool was used for log generation, specifically on the Windows Machine, to collect more detailed logs that reference not only normal alerts or messages from processes and events, but also event timelines and command execution timelines, which help the analyst gain more insight on how certain commands are executed, and if the command was suspicious or justified by parallel processes running that require specific resources. This tool is called Sysmon, a Windows Event Tool, free to use and specifically used in cybersecurity environments to enrich the data with much more exact telemetry.

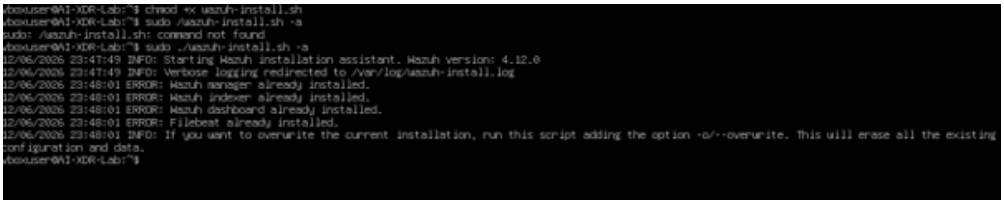

Figure 4.4: Wazuh Installation & Configuration

All of the logs are generated by the Wazuh Agents installed on both machines, which monitor ongoing events and processes, translating them into readable logs, from process execution to

event termination and results. All the generated alerts by Wazuh were selected with the use of predefined rules, where specific information was extracted (timeline of process or event, executed command, time taken to respond, time taken to execute, termination, approval chain). Rather than directly taking the logs from both endpoints, this solution was used to create the standard of logs and keep them readable and understandable by both the AI model and analysts.

Using Flask, an endpoint was exposed for all of the types of generated Wazuh alerts, making it easier to display them on the dashboard.

```
backend-ai-engine > app.py > ...
175
176  @app.route("/api/xdr/wazuh-alerts")
177  def get_xdr_wazuh_alerts():
178
179      reader = WazuhAlertFileReader()
180
181      raw_alerts = reader.get_recent_alerts(limit=20)
182
183      events = WazuhAlertTransformer.transform(raw_alerts)
184
185      ai_results = AIDetectionService.detect_anomalies(events)
186
187      ai_results_by_id = {
188          result["event_id"]: result
189          for result in ai_results
190      }
191
192      response = []
193
194      for event in events:
195
196          event_data = asdict(event)
197
198          score = ThreatScoringService.calculate_score(event)
199
200          classification = DecisionService.classify(score)
201
202          ai_result = ai_results_by_id.get(event.id, {})
203
204          is_anomaly = ai_result.get("is_anomaly", False)
205          anomaly_score = ai_result.get("anomaly_score", 0)
206
207          action = DecisionService.recommend_ai_action(
208              score,
209              is_anomaly,
210              anomaly_score
211          )
212
213          event_data["threat_score"] = score
214          event_data["classification"] = classification
215          event_data["recommended_action"] = action
216          event_data["is_anomaly"] = is_anomaly
```

Figure 4.5: Exposed API Endpoints

In conclusion, Wazuh was one of, if not the integral part of this setup, used not only for log collection, but also for storage and log processing to create a unified source of log generation that can later be modified if more specific event alerts are required for analysis.

The second part of this solution includes a database service, implemented to keep a persistent storage of the extracted and processed logs, in order to optimize the communication between the web-based dashboard and the backend pulling data from the exposed API endpoints.

The selected DB service provider is SQLite, primarily used for the fact that it is a software library running directly inside the solution, rather than a separated network that would require exposing an endpoint to access it, writing data to a file that includes not only the data itself, but also the table structure, or even the relations, indexes and triggers if the schema includes more than one table. In the case of this solution, this schema included only one table, with indexing on columns such as timestamp, endpoint information, event type and classification fields, reducing the redundancy and flooding of the data with unimportant repeated events.

backend-ai-engine > services > database_service.py > DatabaseService > initialize

```python
import sqlite3

DB_NAME = "filtered_logs.db"

class DatabaseService:

    @staticmethod
    def get_connection():
        return sqlite3.connect(DB_NAME)

    @staticmethod
    def initialize():

        conn = DatabaseService.get_connection()

        cursor = conn.cursor()

        cursor.execute("""
        CREATE TABLE IF NOT EXISTS security_events(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            timestamp TEXT,
            endpoint TEXT,
            source_ip TEXT,
            severity TEXT,
            event_type TEXT,
            description TEXT,
            threat_score INTEGER,
            classification TEXT,
            recommended_action TEXT,
            anomaly_score REAL,
            is_anomaly INTEGER,
            UNIQUE(timestamp, endpoint, description, event_type)
        )
        """)

        cursor.execute("""
        CREATE INDEX IF NOT EXISTS idx_timestamp
        ON security_events(timestamp)
        """)

        cursor.execute("""
        CREATE INDEX IF NOT EXISTS idx_endpoint
```

Figure 4.6: Database Service SQLite

4.5 Artificial Intelligence Detection Engine

The AI detection layer is the analytical component of this solution, with its primary goal being the detection of anomalies and abnormalities from the various alerts it receives, as well as helping in the response action decision by providing context from well-known security threats that contain similar occurrences. The structure of this layer consists of two different models, trained with different kinds of data, that work with each other to provide accuracy and efficiency of the system, as well as a rule-based mechanism of classifying severity and impact of a potential threat.

backend-ai-engine > train_model.py > ...

```python
import pandas as pd
from pathlib import Path
from sklearn.ensemble import IsolationForest
import joblib

DATASET_PATH = Path("../dataset/CICIDS2017")

csv_files = list(DATASET_PATH.glob("*.csv"))

monday_file = DATASET_PATH / "Monday-WorkingHours.pcap_ISCX.csv"

df = pd.read_csv(monday_file)
df.columns = df.columns.str.strip()

features = [
    "Flow Duration",
    "Total Fwd Packets",
    "Total Backward Packets",
    "Flow Bytes/s",
    "Flow Packets/s",
    "Fwd Packet Length Mean",
    "Bwd Packet Length Mean",
    "Packet Length Mean"
]

X = df[features]

model = IsolationForest(
    contamination=0.01,
    random_state=42
)

model.fit(X)

print("Model trained successfully")

joblib.dump(model, "models/isolation_forest_model.pkl")

print("Model saved to models/isolation_forest_model.pkl")
```

Figure 4.7: Isolation Forest Algorithm

The first component of this layer is the Isolation Forest model, a tree-based algorithm meant to be trained with un-labelled data, which uses a tree-like structure to detect any outliers from splitting different features of data, and singling out tree nodes that are shorter than the median. The data used to train this model was taken from the Canadian Institute of Cybersecurity Intrusion Detection dataset (CICIDS2017) [41], containing network traffic features with enough context to simulate how an unsupervised learning algorithm can better understand if something is suspicious.

The second component of the layer is a similar tree-based algorithm called Random Forest, a supervised learning algorithm that can only be trained with labelled data. The data used to train this algorithm was taken from generated and processed Wazuh alerts coming from both machines. All the data is labelled with event attributes, such as severity level, threat score, event type and endpoint information, to determine whether an event is abnormal or not.

backend-ai-engine > train_log_model.py > ...

```python
import joblib
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv("dataset/log_training_data_labeled.csv")

print(f"Loaded {len(df)} records")

severity_encoder = LabelEncoder()
event_type_encoder = LabelEncoder()
endpoint_encoder = LabelEncoder()

df["severity_encoded"] = severity_encoder.fit_transform(df["severity"])
df["event_type_encoded"] = event_type_encoder.fit_transform(df["event_type"])
df["endpoint_encoded"] = endpoint_encoder.fit_transform(df["endpoint"])

X = df[
    [
        "severity_encoded",
        "threat_score",
        "event_type_encoded",
        "endpoint_encoded"
    ]
]

y = df["is_anomaly"]

model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    class_weight="balanced"
)

model.fit(X, y)

print("Saving model...")

joblib.dump(model, "models/log_event_model.pkl")
joblib.dump(severity_encoder, "models/severity_encoder.pkl")
joblib.dump(event_type_encoder, "models/event_type_encoder.pkl")
joblib.dump(endpoint_encoder, "models/endpoint_encoder.pkl")
```

Figure 4.8: Random Forest Algorithm

Besides these two models, a rule-based mechanism was also implemented, using keywords that are usually interpreted as suspicious actions such as a PowerShell Command Execution or privilege escalation alert that might insinuate a possible incoming threat. Based on that, the threat score of that event is assigned based on the expected severity.

```python
    @staticmethod
    def local_threat_score(event):
        text = f"{event.event_type} {event.description}".lower()

        if "suspicious file drop" in text:
            return 100

        if "failed login" in text or "bad password" in text:
            return 65

        if "account discovery" in text or "net.exe" in text:
            return 45

        if "powershell" in text:
            return 80

        if "suspicious network connection" in text:
            return 75

        if "successful sudo" in text or "sudo to root" in text:
            return 15

        if "successful login" in text:
            return 10

        if "login session opened" in text or "login session closed" in text:
            return 10

        if event.severity == "Critical":
            return 80

        if event.severity == "High":
            return 60

        if event.severity == "Medium":
            return 35

        if "successful sudo" in text:
            return 50

```

Figure 4.9: Rule-based mechanism of detection

This kind of approach is made so that all events must pass through all layers before being finally labelled as potential threats or not, thus lowering the chance of a false positive or a true negative.

4.6 Threat Scoring & Correlation Layer

After detection, the filtered Wazuh generated alerts go through the threat scoring and rule-based correlation. In this layer, all the filtered alerts are further analysed for any kind of connection between events, either executed in the same endpoint, or in parallel on two or more endpoints, which is essential in detecting scaled attacks that target the entire network.

The first step is assigning a threat score to all events, giving higher scores to events related to suspicious activity such as PowerShell command execution, user privilege access, privilege escalation, suspicious file creation or even abnormal loads of internet activity in out-of-work

hours. After assigning the threat score, all of them are categorized on different levels, with Low being the low risk & impact events, followed by Medium, High and Critical being the highest, much like the CVSS 2.0 standard of threat categorization.

```python
class ThreatScoringService:

    @staticmethod
    def calculate_score(event: SecurityEvent):

        score = 0

        if event.severity == "Critical":
            score += 80
        elif event.severity == "High":
            score += 60
        elif event.severity == "Medium":
            score += 35
        elif event.severity == "Low":
            score += 5

        text = f"{event.event_type} {event.description}".lower()

        if "powershell" in text:
            score += 35

        if "failed login" in text or "logon failure" in text or "bad password" in text:
            score += 30

        if "account discovery" in text or "discovery activity" in text or "net.exe" in text:
            score += 35

        if "successful sudo" in text or "sudo to root" in text:
            return 15

        if "privilege escalation" in text:
            score += 25

        if "suspicious file drop" in text or "executable file dropped" in text:
            score += 40

        if "malware" in text:
            score += 35

        if "suspicious network connection" in text:
            score += 30

        if "successful login" in text:
            score += 5
```

Figure 4.10: Threat Scoring Method

After threat evaluation, the next step is to find whether there is any correlation between events, analysed on different features such as event timestamp and information, providing a clear view of events that alone might be normal but if combined may indicate suspicious activity, all of this being done using a rule-based mechanism. After correlation, additional context is added with the help of the previously used MITRE ATT&CK knowledge base, giving an experienced analyst the needed context to identify the TTPs based on well-known threat actors that operate currently.

```python
class CorrelationService:

    TIME_WINDOW_MINUTES = 30

    @staticmethod
    def parse_timestamp(timestamp):
        try:
            return datetime.strptime(timestamp, "%Y-%m-%dT%H:%M:%S.%f%z")
        except Exception:
            return None

    @staticmethod
    def has_sequence(events, sequence):
        event_types = [event.event_type for event in events]

        position = 0

        for event_type in event_types:
            if event_type == sequence[position]:
                position += 1

                if position == len(sequence):
                    return True

        return False

    @staticmethod
    def correlate(events: list[SecurityEvent]):

        correlated_results = []
        events_by_endpoint = {}

        for event in events:
            events_by_endpoint.setdefault(event.endpoint, []).append(event)

        for endpoint, endpoint_events in events_by_endpoint.items():

            endpoint_events = sorted(
                endpoint_events,
                key=lambda event: CorrelationService.parse_timestamp(event.timestamp)
                or datetime.min
            )

            event_types = [event.event_type for event in endpoint_events]
```

Figure 4.11: Rule-Based Correlation

After passing through both components, the output of these alerts is stored along with the correlation score, other related events, identified attack stages (results may not be accurate) and recommended response actions, which will be used in the response layer to help determine the kind of response needed to be implemented.

4.7 Decision & Response Engine

The Decision & Response Engine is the last step of the simulated XDR system, representing the action taken by the system when facing a threat. Previous sections have covered the threat scoring & additional context per event and threat categorization, as well as suggested response actions. All of these are taken into account by the decision engine, which further analyses the outcomes, and determines the course of action for a specific event, or for a few correlated suspicious events.

After threats are given the initial score and then categorized, they are prioritized, with low-risk threats being marked as normal machine activity, medium being kept under further surveillance for future similar events, and high & critical following actions such as threat escalation and investigation or even extreme measures such as endpoint isolation from the network.

Events that are already marked as suspicious by the AI detection layer are considered more

important to deal with, and take precedence over the other security events, skipping the rule-based mechanism that determines the severity and impact that these events might have, and taking proper action.

```
43      @staticmethod
44
45      def recommend_correlation_action(correlation_score):
46
47          if correlation_score >= 85:
48              return 'Simulate Endpoint Isolation'
49
50          if correlation_score >= 60:
51              return 'Escalate Incident'
52
53          if correlation_score >= 30:
54              return 'Generate Alert'
55
56          return 'Continue Monitoring'
```

Figure 4.12: Recommended Response Rules

It should be noted that these responses are generated by the system, but there are no actual actions taken against the machines producing the logs, which can be a significant addition to the build-up of this thesis. However, this should still be supervised by a human, as the data that the detection models have been trained with, and the rule-based mechanisms, might not be fully inclusive or absolute, making the system obsolete against newer threats at least until an adapted LLM for security events trained with contemporaneous data is established.

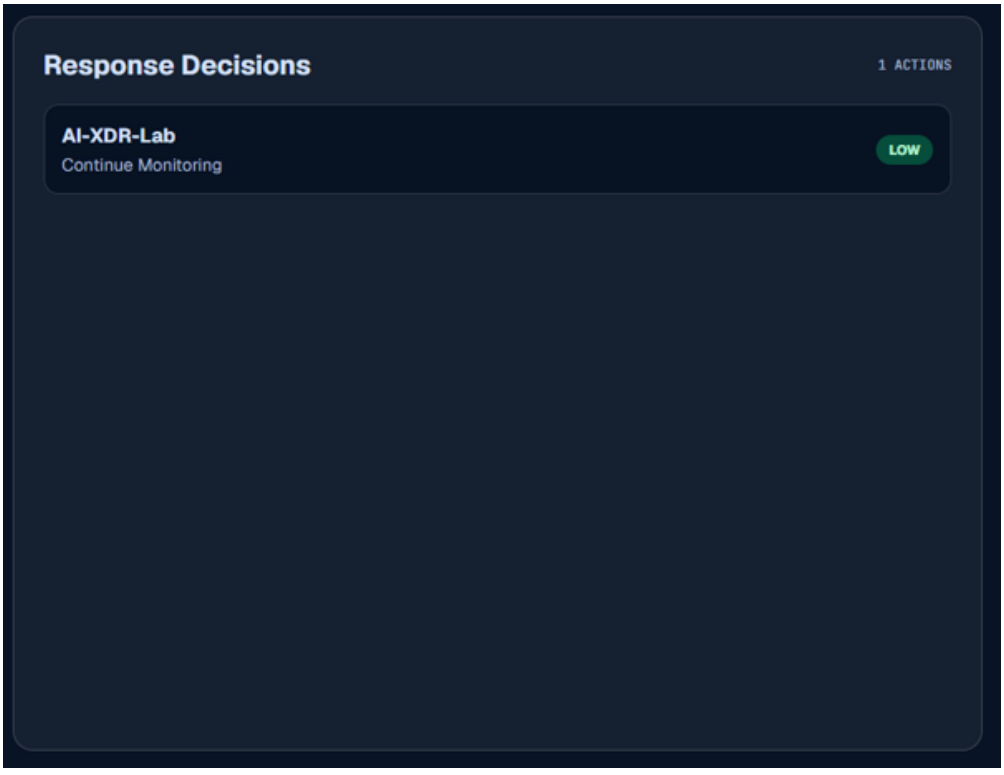

Figure 4.13: Response Decisions from Rule-based Mechanism

4.8 Web-based Dashboard Implementation

The web-based Dashboard is the only part of this system that the user (analyst) can interact with. It was developed using React & TypeScript as mentioned, displaying all the information taken from REST API endpoints created through Python's Flask, with main content taken from the database source implemented with SQLite.

It has four different sections, each representing a functionality that current XDR solutions possess, making it a perfect simulated interactable system for analysts:

- Overview Dashboard Page
- Log History
- Threat Analysis & Correlation
- System Status

The overview dashboard displays data cards that visualize the latest pulled events from Wazuh Agents, critical or important alerts, active endpoints and most importantly the decision & response actions that have been simulated.

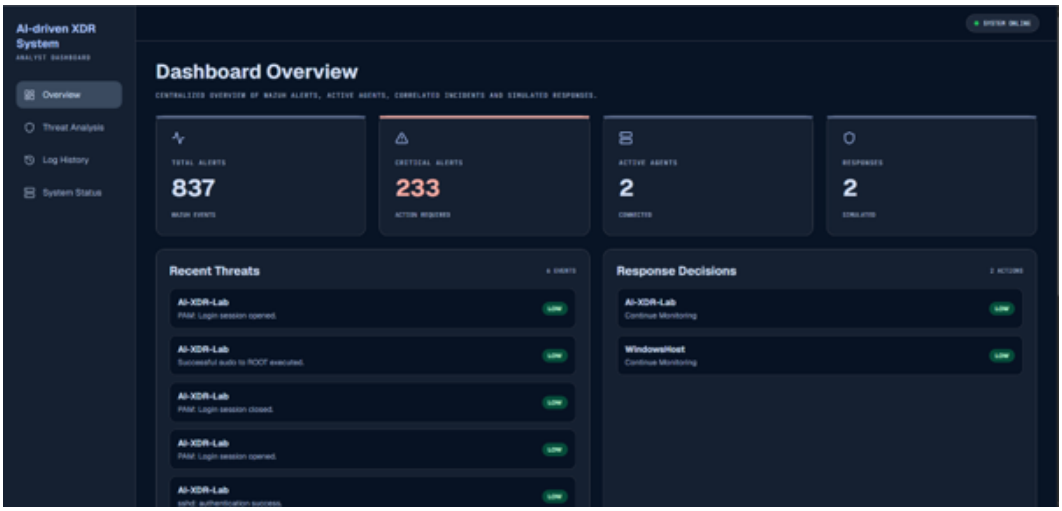

Figure 4.14: Overview Dashboard

The threat analysis page displays the correlation score results, enriched with MITRE ATT&CK TTPs that correspond to the correlated events and recommended response actions.

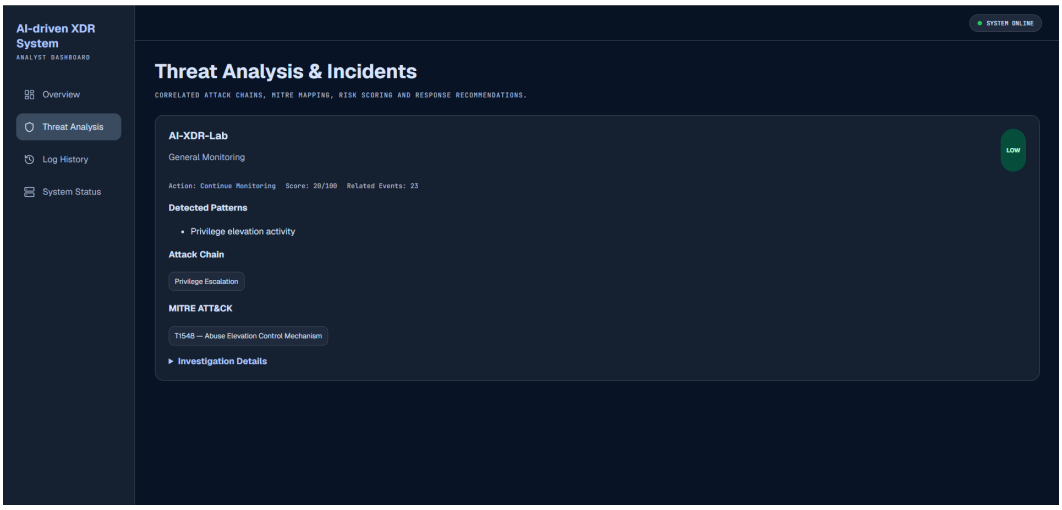

Figure 4.15: Threat Analysis Page

The log history page displays a detailed view of all the logs, that analysts can use to further analyse all the events one by one manually, or understand how the AI detection service executed its analysis of the logs. All the logs are filterable and indexable, making it easier for the analyst to trace specific logs, or batches of logs coming from a machine, and the ability to extract the filtered data for other external tools to analyse these suspicious events.

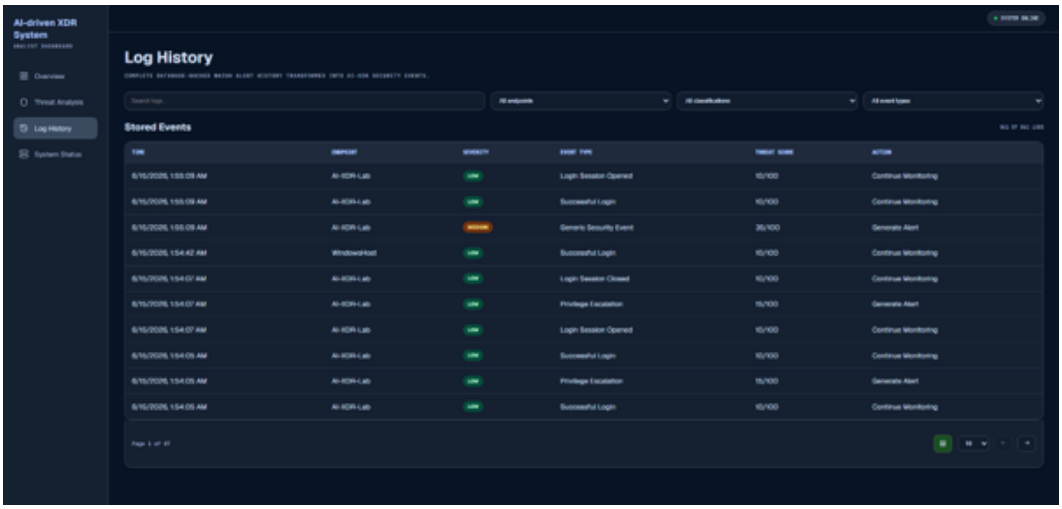

Figure 4.16: Log History Page

The system status page is mainly used for testing purposes, displaying a live connection update to all the layers that are part of this system, as well as the machines feeding the logs.

Figure 4.17: System Status Page

4.9 Testing & Validation

The last section of this chapter covers testing and validation of the implemented XDR system, focusing on the way it processes logs and the accuracy of the implemented ML algorithms used for detection, and the effectiveness of the rule-based mechanisms that incur predefined rules when identifying threat potential, assigning score and finding correlation between events & logs.

The two models used for the AI Detection layer were Isolation Forest and Random Forest, as mentioned in the previous sections. For Isolation Forest, there is no precise metric used to measure the accuracy of this component, but for Random Forest, a supervised learning algorithm, the metrics can be generated, keeping in mind that this layer has been trained with data collected by the two endpoints that this XDR system is connected to. The dataset is smaller than the one used for Isolation Forest, with an approximate 400 records used to train this dataset, but with continuous telemetry generation, the model can be adjusted and provide accurate results even with large amounts of data.

```
(venv) PS C:\Users\Pc\Desktop\AI-XDR-Simulation\AI_XDR_SIM\backend-ai-engine> python train_log_model.py
Loaded 400 records
['timestamp', 'endpoint', 'event_type', 'severity', 'classification', 'threat_score', 'source_ip', 'recommended_action', 'description', 'is_anomaly']

============================================================
 AI-XDR RANDOM FOREST EVALUATION REPORT
============================================================
Dataset Size      : 400 records
Normal Events     : 243
Anomalous Events  : 157

Performance Metrics
------------------------------------------------------------
Accuracy          : 98.75%
Precision         : 96.88%
Recall            : 100.00%
F1 Score          : 98.41%

Confusion Matrix
------------------------------------------------------------
True Negatives    : 48
False Positives   : 1
False Negatives   : 0
True Positives    : 31
============================================================
Saving model...
Training completed successfully.
Samples used: 400
Normal events: 243
Anomalous events: 157
```

Figure 4.18: Metrics for trained Random Forest Model

Testing Scenario 1 — Successful Login Attempt (Windows Machine)

Test Description & Expected Results: The user successfully logs in on the designated endpoint, and the AI-driven detection layer correctly classifies it as a Low-Risk event.

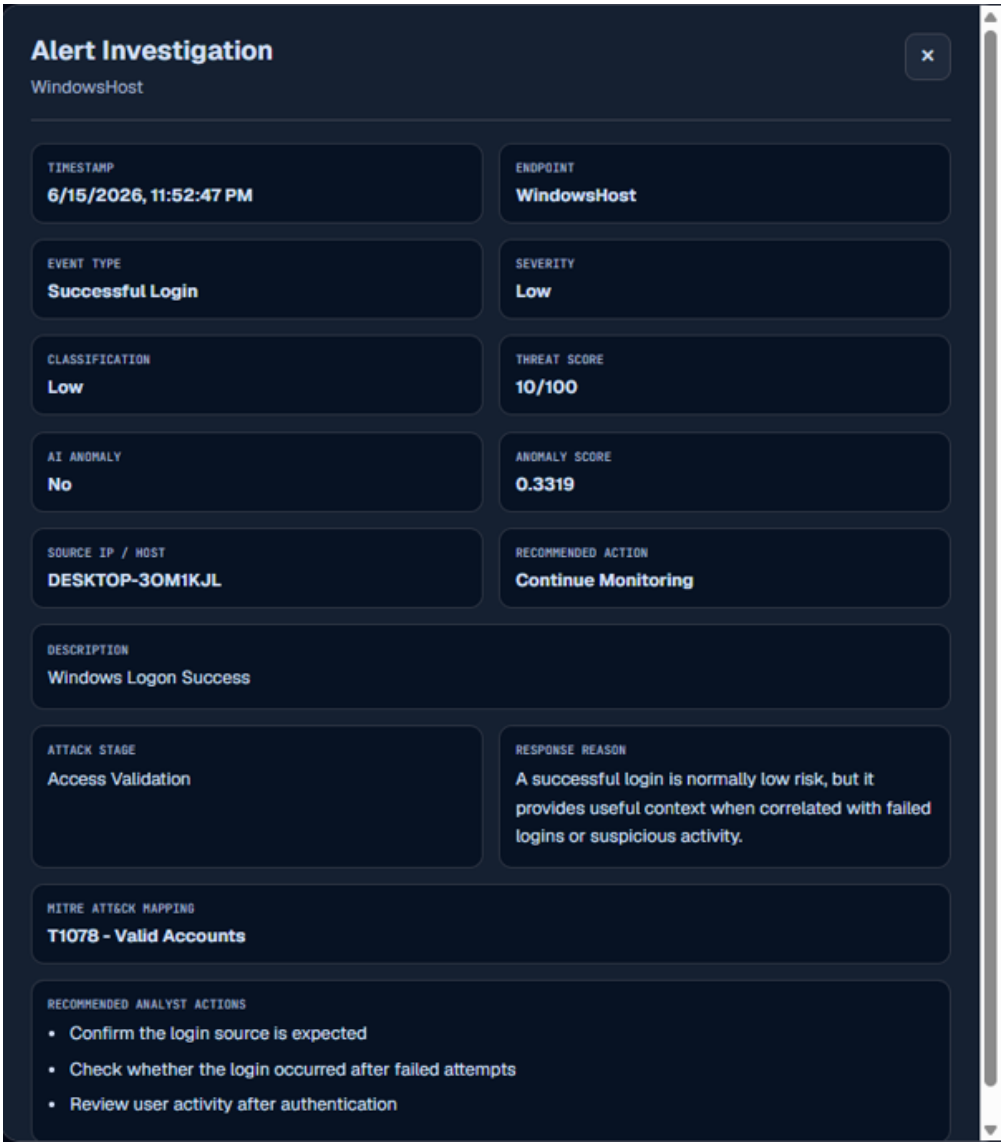

Figure 4.19: Low Level Scoring by the AI Detection layer

Testing Scenario 2 — Numerous Consequential Invalid Login Attempts (Windows Machine)

Test Description & Expected Results: The user fails to log in numerous times on the machine due to invalid credentials; the AI-driven detection layer correctly classifies it as a High-Risk event, indicating escalation and endpoint monitoring.

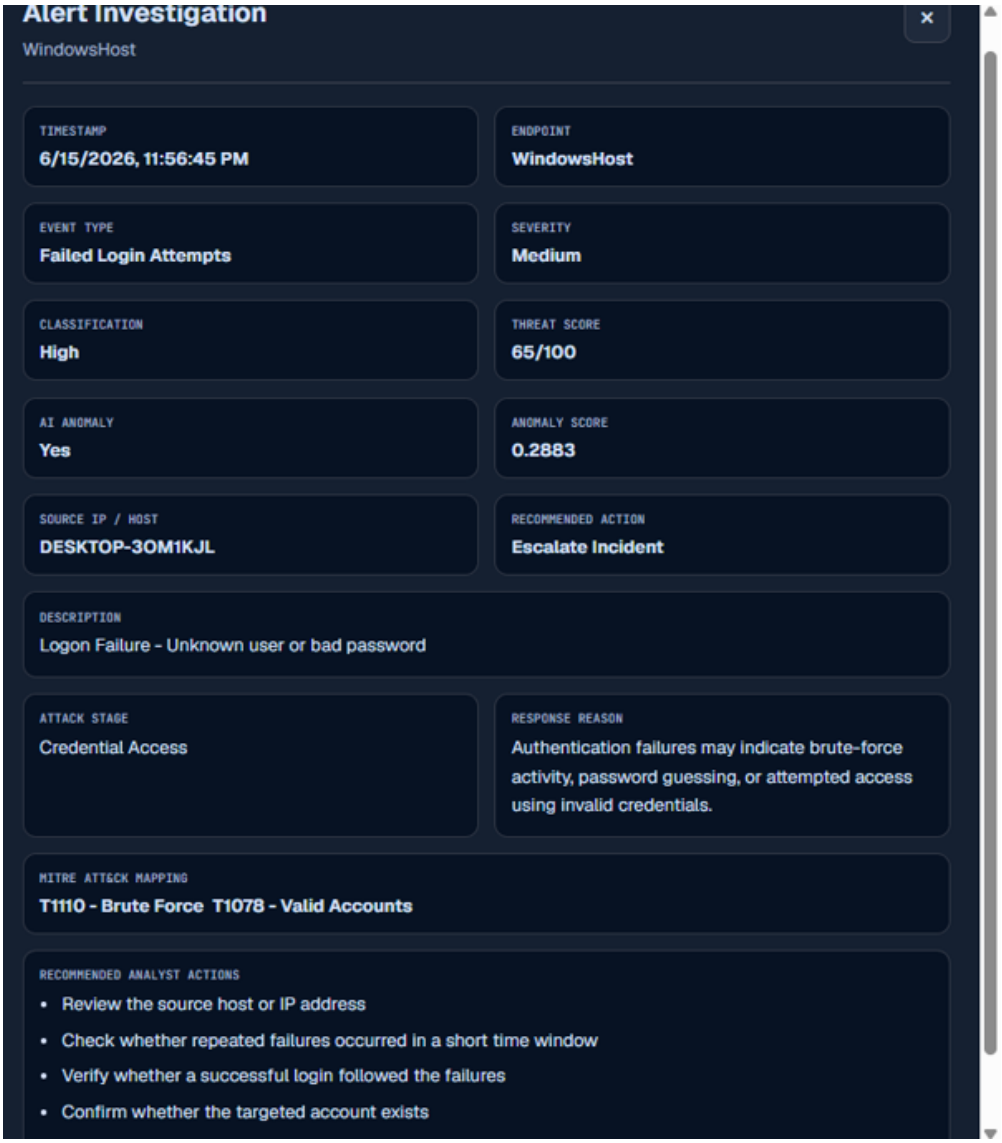

Figure 4.20: AI-driven Detection layer correctly predicting anomaly on login attempts

Testing Scenario 3 — PowerShell Command Execution (Windows Machine)

Test Description & Expected Results: Executed PowerShell commands that explore access and privileges of the current logged-in user should be indicated and marked as potentially suspicious, following with further escalation and monitoring.

Executed Commands:

- `whoami`
- `powershell -ExecutionPolicy Bypass -Command "Get-Process"`
- `powershell -ExecutionPolicy Bypass -EncodedCommand SQBFAFgA`

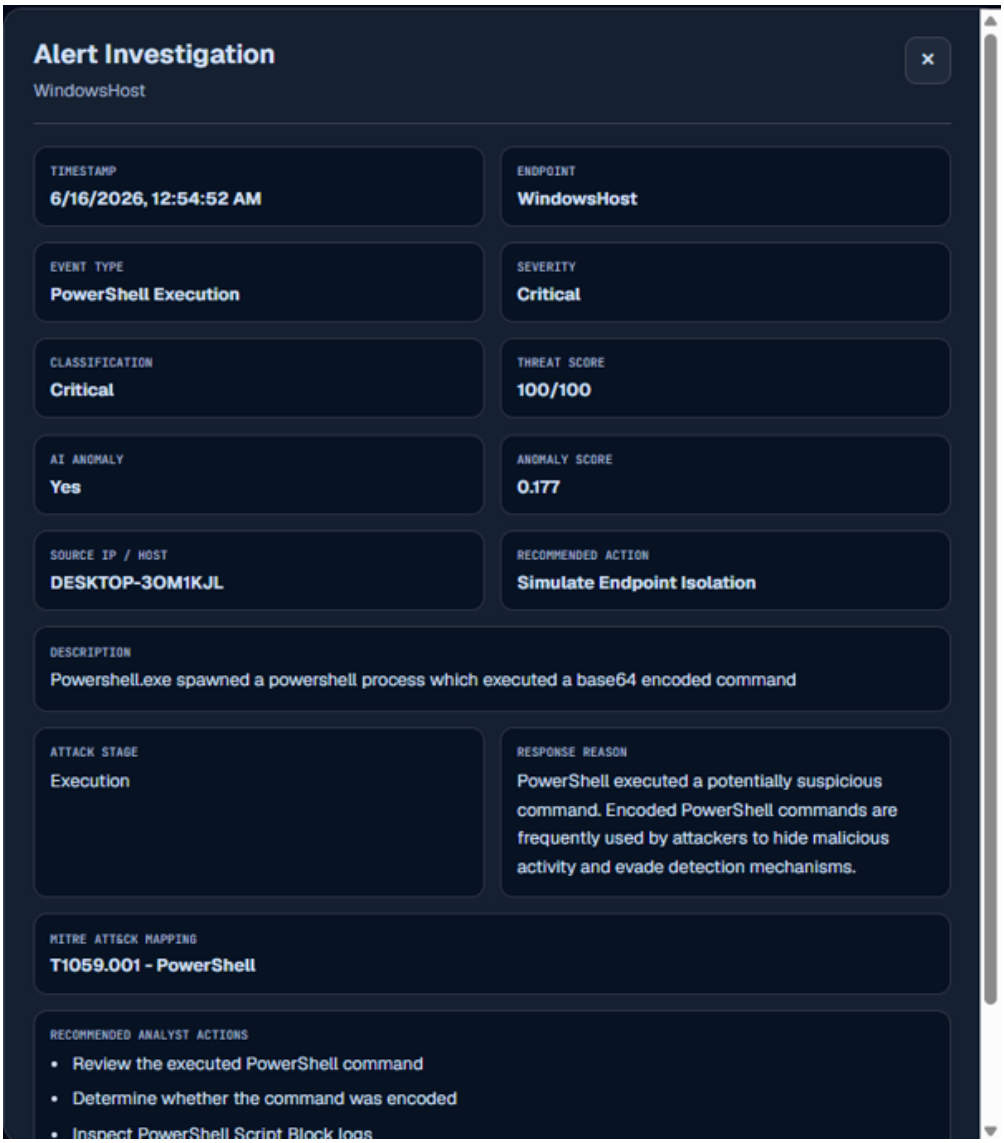

Figure 4.21: Encoded Command Execution (Log successfully detected as suspicious)

Testing Scenario 4 — Privilege Escalation, Higher Threat Score (Ubuntu Machine)

Test Description & Expected Results: Execution of privilege escalation commands on the Ubuntu Machine should indicate that the logged-in user tried to gain knowledge on its privileges and access to the system.

Executed Commands:

- `sudo whoami`
- `sudo tail -n 20 /var/log/auth.log`

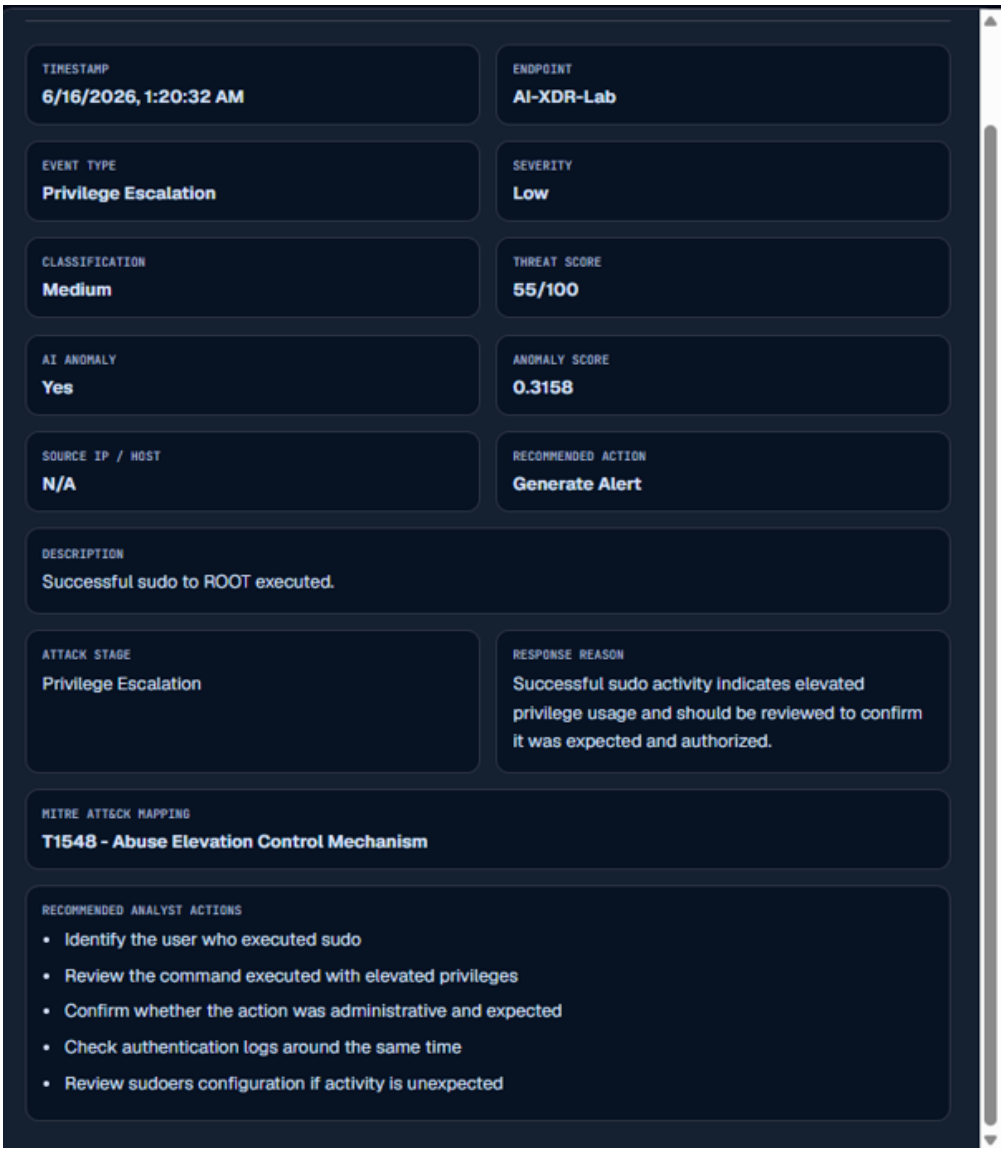

Figure 4.22: Privilege Escalation Ubuntu Machine

Chapter 5 – CONCLUSION

This thesis was aimed at giving an implementation of an AI-driven XDR simulated system. The proposed solution integrates all the components and elements that current XDR systems offer (log collection and centralization, event processing and so on), but one thing that not all systems have adapted is the AI anomaly detection layer, an upgrade to the usual rule-based mechanisms that these current systems use to autonomously detect anomalies. The simulated system was established through the use of a Windows and Ubuntu Machine, which helped a lot with log generation, and after being deployed, was tested for test scenarios in a white box testing manner, where it successfully detected anomalies and assigned appropriate responses to them. All the functionalities that this system offers are built in a component-like manner, making every part of this system autonomous and easily scalable if future fine tuning needs to be done. It combines AI, ML and rule-based mechanisms to create an XDR solution, solid enough to detect any anomalies that the network's endpoints encounter, correlating and cross-referencing threat patterns with known TTPs from MITRE ATT&CK, as well as simulated responses that should be applied to the endpoint/s.

The integration of these ML algorithms helped to shape the AI-driven detection component, where the Random Forest classifier achieved optimal performance metrics, with an accuracy of 98.75%, a precision of 96.88% and a F1-score of 98.41%. Meanwhile the Isolation Forest model provided the analysis of outliers, something that this algorithm is specifically focused on, to detect anomalies and output them with specific impact levels.

The implementation phase also provided an answer to the research questions posed about the response layer. This layer generated responses based on context given by other layers, making it a perfect demonstration of how security solutions operate in real-world conditions.

Another research question that was answered was the functionality provided by the event correlation layer, which successfully found links between events and visualized possible attack chains on the network.

Many existing academic works focus on anomaly detection and event correlation, but purely rule-based. However, the AI-driven detection layer offers more coverage for known threats, with training happening continuously, where the database is updated and all the logs can be re-used to train the model, along with existing robust datasets.

Despite the success of the implementation of this system, there are certain limitations worth mentioning that might not fully replicate the XDR solutions that are commonly used. One of them being the complexity and magnitude of the usual XDR solutions, that monitor a large number of endpoints, by integrating and correlating events between this large number

of endpoints, making the systems even more complex. Another limitation is the datasets, as mentioned in the Literature Review chapter, the dataset that was used to train one of the models was relatively small, because this dataset was created by generated logs from this system's connected endpoints. Lastly, the response actions that are decided for a specific anomaly detected are not executable and cannot access the endpoint directly, but rather simulate the process of assigning this response to that endpoint by labelling the action and the course of reason.

However, these limitations only leave room for future improvements, starting with more enriched datasets that can be used to fine-tune the models for better accuracy of detection. The response layer of this solution could be able to directly access the endpoint and execute the response action appropriately, as well as providing better context for correlation between events. Another important improvement of this thesis would be to containerize and ship this implemented system as one package using services like Docker, to add support for cloud-based environments.

Bibliography

[1] MITRE ATT&CK, "Phishing (T1566)," *MITRE ATT&CK®*. [Online]. Available: https://attack.mitre.org/techniques/T1566/ [Accessed: Mar. 16, 2026].

[2] National Institute of Standards and Technology, *Computer Security Incident Handling Guide*, NIST Special Publication 800-61r3. Available: https://doi.org/10.6028/NIST.SP.800-61r3

[3] P. Paidy, "Unified Threat Detection Platform with AI, SIEM, and XDR," *International Journal of Advanced Intelligence in Data Science and Machine Learning (IJAIDSML)*, vol. 6, no. 1, pp. 95–104, Jan. 2025, doi: https://doi.org/10.63282/3050-9262.IJAIDSML-V6I1P111.

[4] N. N. A. Sjarif, S. Chuprat, M. N. Mahrin, N. A. Ahmad, A. Ariffin, and F. M. Senan, "Endpoint Detection and Response: Why Use Machine Learning?," *IEEE Xplore*, DOI: 10.1109/ICOIN50798.2020.8939836.

[5] Microsoft, "What is XDR?," *Microsoft Security*. [Online]. Available: https://www.microsoft.com/en-us/security/business/security-101/what-is-xdr [Accessed: May 20, 2026].

[6] D. Pissanidis and K. Demertzis, "Integrating AI/ML in Cybersecurity: An Analysis of Open XDR Technology and Its Application in Intrusion Detection and System Log Management," *Preprints*, Dec. 2023, doi: 10.20944/preprints202312.0205.v1.

[7] MITRE ATT&CK, "MITRE ATT&CK Framework," *MITRE ATT&CK®*. [Online]. Available: https://attack.mitre.org/ [Accessed: May 20, 2026].

[8] N. Mohamed, "Artificial intelligence and machine learning in cybersecurity: a deep dive into state-of-the-art techniques and future paradigms," *Knowledge and Information Systems*, vol. 67, pp. 6969–7055, 2025. https://doi.org/10.1007/s10115-025-02429-y

[9] S. Survaiya, V. Uke, and I. Vighe, "AI in cybersecurity: Intrusion detection system," *International Advanced Research Journal in Science, Engineering and Technology*, vol. 12, no. 11, 2025, doi: 10.17148/IARJSET.2025.121106.

[10] N. Mohamed, "Current trends in AI and ML for cybersecurity: A state-of-the-art survey," *Cogent Engineering*, vol. 10, no. 2, p. 2272358, 2023, DOI: 10.1080/23311916.2023.2272358.

[11] S. Survaiya, V. Uke, and I. Vighe, "AI in cybersecurity: Intrusion detection system," *International Advanced Research Journal in Science, Engineering and Technology*, vol. 12, no. 11, Nov. 2025, doi: 10.17148/IARJSET.2025.121106.

[12] D. Tomar, K. Chhillar, and D. Kumhar, "Comparative study of supervised, unsupervised and reinforcement learning approaches for malware detection," *International Journal of Innovative Research in Technology (IJIRT)*, vol. 12, no. 4, pp. 2793–2801, Sep. 2025.

[13] Canadian Institute for Cybersecurity, "CICIDS2017 Dataset." [Online]. Available: https://www.unb.ca/cic/datasets/ids-2017.html [Accessed: May 20, 2026].

[14] S. T. Erukude, V. C. Marella, and S. R. Veluru, "AI-Driven Cybersecurity Threats: A Survey of Emerging Risks and Defensive Strategies," arXiv:2601.03304, Jan. 2026. [Online]. Available: https://arxiv.org/abs/2601.03304

[15] J. Pan, "AI based Log Analyser: A Practical Approach," arXiv, 2022.

[16] NSA & CISA, "Implementing SIEM and SOAR Platforms Guidance," 2025.

[17] A. Ehis, "Optimization of SIEM Infrastructures and Event Correlation Analysis," 2023.

[18] N. Sharma and N. Jindal, "Emerging artificial intelligence applications: metaverse, IoT, cybersecurity, healthcare – an overview," *Multimedia Tools and Applications*, vol. 83, no. 19, pp. 57317–57345, 2024.

[19] C. Kyrkou et al., "Towards artificial-intelligence-based cybersecurity for robustifying automated driving systems against camera sensor attacks," in *Proc. 2020 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2020.

[20] S. Dua and X. Du, "Data Mining and Machine Learning in Cybersecurity," arXiv:1904.03491, 2019. [Online]. Available: https://arxiv.org/abs/1904.03491

[21] N. Mohamed, "Current Trends in AI and ML for Cybersecurity: A State-of-the-Art Survey," *Cogent Engineering*, 2023.

[22] I. Kim et al., "Toward Robust Security Orchestration and Automated Response," *Information*, 2025.

[23] M. S. Arsanjani, "The Anatomy of Agentic AI," *Medium*, 2023. [Online]. Available: https://dr-arsanjani.medium.com/the-anatomy-of-agentic-ai-0ae7d243d13c [Accessed: May 19, 2026].

[24] N. Katiyar, "AI And Cyber-Security: Enhancing Threat Detection And Response With Machine Learning," *Educational Administration: Theory and Practice*, vol. 30, no. 4, pp. 6273–6282, 2024. doi: 10.53555/kuey.v30i4.2377.

[25] G. Palma, G. Cecchi, M. Caronna, and A. Rizzo, "Leveraging Large Language Models for Scalable and Explainable Cybersecurity Log Analysis," *Journal of Cybersecurity and Privacy*, vol. 5, p. 55, 2025. https://doi.org/10.3390/jcp5030055

[26] Skedler, "What is Explainable AI (XAI)?" [Online]. Available: https://www.skedler.com/blog/what-is-explainable-ai-xai/ [Accessed: May 19, 2026].

[27] A. Vassilev, A. Oprea, A. Fordyce, and H. Anderson, *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*, NIST AI 100-2e2023, 2024. https://doi.org/10.6028/NIST.AI.100-2e2023

[28] Simplilearn, "CIA Triad in Cyber Security," [Online]. Available: https://www.simplilearn.com/cia-triad-in-cyber-security-article [Accessed: May 19, 2026].

[29] G. Sarraf and V. Pal, "Autonomous Threat Detection and Response in Cloud Security: A Comprehensive Survey of AI-Driven Strategies," 2025.

[30] V. Shewale, "Beyond EDR: Exploring the rise of XDR for unified threat detection and response," *World Journal of Advanced Engineering Technology and Sciences*, vol. 15, no. 02, pp. 380–386, 2025. doi: https://doi.org/10.30574/wjaets.2025.15.2.0551

[31] Microsoft, "What is the MITRE ATT&CK Framework?" *Microsoft Security*. [Online]. Available: https://www.microsoft.com/en-us/security/business/security-101/what-is-mitre-attack-framework [Accessed: May 20, 2026].

[32] IBM, "What is the MITRE ATT&CK Framework?" *IBM Think*. [Online]. Available: https://www.ibm.com/think/topics/mitre-attack [Accessed: May 20, 2026].

[33] MITRE Corporation, "ATT&CK," *MITRE ATT&CK*. [Online]. Available: https://attack.mitre.org/ [Accessed: May 20, 2026].

[34] National Institute of Standards and Technology (NIST), "Identify, Protect, Detect, Respond and Recover: The NIST Cybersecurity Framework," *NIST Taking Measure Blog*. [Online]. Available: https://www.nist.gov/blogs/taking-measure/identify-protect-detect-respond-and-recover-nist-cybersecurity-framework [Accessed: May 20, 2026].

[35] IBM, "ISO 27001 Compliance – IBM Cloud," *IBM Products*. [Online]. Available: https://www.ibm.com/products/cloud/compliance/iso-27001 [Accessed: May 20, 2026].

[36] X. Li et al., "Explainable dimensionality reduction (XDR) to unbox AI 'black box' models," *Humanities and Social Sciences Communications*, vol. 10, p. 35, 2023. https://doi.org/10.1057/s41599-023-01505-4

[37] N. H. A. Mutalib et al., "Explainable deep learning approach for advanced persistent threats (APTs) detection in cybersecurity: a review," *Artificial Intelligence Review*, vol. 57, p. 297, 2024. https://doi.org/10.1007/s10462-024-10890-4

[38] A. S. George, A. George, Dr. Baskar, and D. Pandey, "XDR: The Evolution of Endpoint Security Solutions," *International Journal of Advanced Research in Science Communication and Technology*, vol. 8, pp. 493–501, 2021. doi: 10.5281/zenodo.7028219.

[39] J. Jana, "The AI Black Box: What We're Still Getting Wrong About Trusting Machine Learning Models," *Hyperight*, Mar. 3, 2025. [Online]. Available: https://hyperight.com/ai-black-box-what-were-still-getting-wrong-about-trusting-machine-learning-models/ [Accessed: May 11, 2026].

[40] A. Hui, S. Ahn, C. Lye, and J. Deng, "Ethical Challenges of Artificial Intelligence in Health Care: A Narrative Review," *Ethics in Biology, Engineering and Medicine*, vol. 12, 2022. doi: 10.1615/EthicsBiologyEngMed.2022041580.

[41] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proc. 4th International Conference on Information Systems Security and Privacy (ICISSP)*, Portugal, January 2018.